

CODEGUARD AI

Agentic Code Review & Test Assistant

MASTER PROJECT GUIDE / PROJECT SHEET

C# / .NET 10 LTS · ASP.NET Core Web API · PostgreSQL · EF Core · Gemini · MCP

Dokümanın amacı

28 günlük kontrollü geliştirme boyunca yalnız ne yapılacağını değil; neden yapıldığını, mimaride nerede durduğunu, nasıl doğrulandığını ve yanlış yapılırsa hangi riskin doğacağını öğretmek.

Proje seviyesi	Staj seviyesinin biraz üzerinde, tek kişiyle gerçekleştirilebilir
Mimari	Modüler monolit; dört ana proje + iki test projesi
Çalışma modeli	Local Windows; dotnet run; local PostgreSQL; dotnet test
Kesin sınırlar	Docker/container yok; Python yok; microservice yok; otonom kod değiştirme yok
Sürüm	1.0 - 29 Eylül 2026

İçindekiler

Başlıklar PDF okuyucusundaki yer imleriyle de gezilebilir.

MASTER PROJECT GUIDE / PROJECT SHEET	1
C# / .NET 10 LTS · ASP.NET Core Web API · PostgreSQL · EF Core · Gemini · MCP	1
İçindekiler	2
1. Proje Kısaca Nedir?	5
2. Projenin Çözdüğü Problem	5
3. Hedef Kullanıcı	5
4. Kullanıcı Senaryosu	5
5. Sistem Nasıl Çalışır?	6
6. Teknoloji Stack'i	7
7. Neden C# / ASP.NET Core?	10
8. Modüler Mimari	10
9. Backend Mimarisi ve Bağımlılık Yönü	12
10. Domain Tasarımı	12
11. Database Tasarımı	12
12. EF Core Yapısı	13
13. AI / LLM Mimarisi	14
14. Structured Output	14
15. Model ve AI Kuralları	14
System Prompt, User Prompt, Context, Tool Result, Response	15
16. Agent Mimarisi	15
17. Review Agent	15
18. Test Agent	15
19. Orchestrator	15
20. Tool Layer	16
21. MCP Mimarisi	16
22. Güvenlik Kuralları	17
23. Ana Özellikler	17

24. API Haritası	18
25. Testing Stratejisi	19
26. LLM Evaluation Stratejisi	19
27. Logging / Observability	19
28. Git / GitHub Stratejisi	19
29. GitHub Actions CI	19
30. Geliştirme Fazları	20
31. 28 Günlük Ayrıntılı Geliştirme Planı	21
31.1 Codex Genel Uygulama Kuralları	22
31.2 Configuration, Options ve Hardcode Politikası	23
31.3 Codex Çıktı Kontrol Şablonu	23
GÜN 1 - Kapsam ve başarı ölçütleri	24
GÜN 2 - Mimari ve solution iskeleti	25
GÜN 3 - API composition root ve health	26
GÜN 4 - HTTP sözleşmesi, validation ve hata modeli	27
GÜN 5 - Configuration ve secret güvenliği	28
GÜN 6 - Domain entity ve enumlar	29
GÜN 7 - DbContext ve EF mappings	30
GÜN 8 - Migration ve local PostgreSQL kurulumu	31
GÜN 9 - Project CRUD ve persistence testleri	32
GÜN 10 - Scanner sözleşmeleri ve manifest	33
GÜN 11 - Path, reparse point ve secret güvenliği	34
GÜN 12 - Safe file reader ve context builder	35
GÜN 13 - ILLMProvider ve fake provider	36
GÜN 14 - Prompt templates ve schema modelleri	37
GÜN 15 - GeminiProvider HTTP adaptörü	38
GÜN 16 - Review Agent ve grounding validator	39
GÜN 17 - Review orchestrator ve review API	40
GÜN 18 - Test Agent structured output	41
GÜN 19 - Tool contracts ve audit envelope	42

GÜN 20 - Safe dotnet test runner	43
GÜN 21 - Test orchestration ve API	44
GÜN 22 - MCP server adapter başlangıcı	45
GÜN 23 - run_tests MCP tool ve sınırlar	46
GÜN 24 - Deterministik orchestrator hardening	47
GÜN 25 - Basit frontend ve demo akışı	48
GÜN 26 - Eval ve observability	49
GÜN 27 - GitHub Actions ve tam doğrulama	50
GÜN 28 - Dokümantasyon, demo ve portföy kapanışı	51
32. Proje Tamamlanma Kriterleri	52
33. Final Ürün ve Teknik Demo Flow	52
34. GitHub Repository Checklist	53
35. CV'de Nasıl Gösterilebilir?	53
36. Future Improvements	53
37. Son Teknik Tutarlılık Kontrolü	54
38. Kaynaklar ve Sürüm Notu	54

1. Proje Kısaca Nedir?

CodeGuard AI, kullanıcının kaydettiği yerel bir kaynak kod repository'sini güvenli kurallarla tarayan, seçilmiş bağlamı Gemini'ye yapılandırılmış biçimde analiz ettiren, doğrulanmış bulguları PostgreSQL'de saklayan ve bu bulgulardan test senaryoları üreten bir developer tool'dur. Chatbot değildir: merkezinde konuşma değil, Project -> Scan -> ReviewRun -> Finding -> TestCase yaşam döngüsü vardır.

Tek cümlelik değer önerisi

CodeGuard AI, LLM çıktısını serbest metin olmaktan çıkarıp dosya ve satır kanıtına bağlı, şema doğrulamalı, geçmişti tutulabilir bir code review kaydına dönüştürür.

2. Projenin Çözdüğü Problem

- Kod inceleme araçları yeni geliştirici için pahalı veya karmaşık olabilir.
- Bir LLM'e bütün repository'yi göndermek secret, maliyet ve bağlam taşması riski üretir.
- Serbest metin tavsiyeleri filtrelenemez, doğrulanamaz ve geçmişle karşılaştırılmaz.
- Modelin uydurduğu dosya/satır bilgisi gerçek bulgu gibi saklanabilir.
- Modelin doğrudan shell veya filesystem erişimi command injection ve veri sızıntısı doğurur.

Çözüm; scanner, context builder, provider abstraction, agent, grounding validator, persistence ve safe tool katmanlarını birbirinden ayırır. Böylece her risk ayrı noktada test edilir.

3. Hedef Kullanıcı

Birincil kullanıcı yerel kodunu analiz etmek isteyen geliştiricidir. Portföy bağlamında ikinci kullanıcı ise projeyi değerlendiren teknik mülakatçı veya staj ekibidir. Ürün, tek kullanıcılı local-first bir MVP olarak tasarlanır; kimlik doğrulama ve çok kiracılı yapı bu sürümde yoktur.

4. Kullanıcı Senaryosu

1. Kullanıcı project adı ve local repository path'i ile kayıt oluşturur.
2. Scan başlatır; included/skipped dosyaları ve nedenlerini görür.
3. Review başlatır; sistem bounded context oluşturur ve Review Agent'ı çalıştırır.
4. Bulgular severity, category, file, line, reason, suggestion ve confidence ile görünür.
5. Kullanıcı review geçmişine dönebilir ve seçilen review için test önerisi üretebilir.
6. Test çalıştırma ancak ayrı, açık kullanıcı aksiyonuyla güvenli runner üzerinden yapılır.

5. Sistem Nasıl Çalışır?

Kullanıcı + Basit Web UI

↓

ASP.NET Core Controllers

↓

Application Services / Orchestrator

↓

Safe Repository Scanner

↓

Context Builder

↓

Review Agent

↓

ILLMProvider -> GeminiProvider

↓

Schema + Grounding Validation

↓

EF Core -> PostgreSQL

↓

REST Response -> UI

Düzeltilmiş test akışı

MCP, agent ile tool arasındaki zorunlu iç bağlantı değildir. Önce Test Agent mevcut Application tool interface'lerini kullanır. MCP daha sonra aynı güvenli implementasyonları dış istemcilere standart protokolle sunan adapter olur. Bu ayrım MCP'yi güvenlik katmanı sanma hatasını önler.

6. Teknoloji Stack'i

Aşağıdaki matris seçimin yalnız adını değil, projedeki rolünü ve trade-off'unu gösterir. Tablo yatay sayfalardadır.

Seçim ilkesi

Her teknoloji doğrudan bir probleme bağlanır: HTTP sınırı, ilişkisel veri, güvenli dış servis çağırısı, doğrulanabilir AI çıktısı veya otomatik kalite kontrolü. Yalnız CV'de görünmesi için teknoloji eklenmez.

İlke	Bu projedeki kararlar
Local-first ve ücretsiz	.NET, local Windows PostgreSQL, Git ve GitHub ücretsiz kullanım olanakları
Az bağımlılık	Built-in DI, System.Text.Json, DataAnnotations ve Microsoft.Extensions.Logging
Değiştirilebilir dış servis	Gemini yalnız ILLMProvider arkasındaki Infrastructure adapterıdır
Test edilebilirlik	FakeLLMProvider, fake HTTP handler ve WebApplicationFactory
Güvenlik sınırı	Scanner ve tool layer modelden bağımsız policy uygular
Kapsam disiplini	Modüler monolit; Docker, microservice ve message broker yok

Sonraki iki yatay sayfa, teknoloji karar kayıtlarının özet kataloğudur. Faz numarası, teknolojinin 28 günlük planda ilk kez ne zaman devreye girdiğini gösterir.

C#	Dil	Statik tipli, modern .NET dili.	Domain, API, servis ve test kodu.	Tip güvenliği ve güçlü araç desteği.	Tüm solution	Java, TypeScript	ASP.NET ekosistemiyle doğal uyum.	1	Zorunlu	async/await, LINQ, records, nullable reference types
.NET 10 LTS	Runtime/SDK	C# uygulamasını derleyen ve çalıştıran desteklenen platform.	Build, runtime ve temel kütüphaneler.	2028'e kadar LTS; Eylül 2026'da aktif destek.	Tüm solution	.NET 8 LTS, .NET 9	Yeni proje için güncel LTS; EF Core 10 ile eşleşir.	1	Zorunlu	SDK, runtime, target framework, LTS
ASP.NET Core Web API	Backend	HTTP API geliştirme frameworkü.	REST endpointleri, DI, middleware, hata modeli.	Performanslı, test edilebilir ve C# odaklı.	CodeGuardAI.Api	Minimal API, MVC	Controller yaklaşımı büyüyen endpoint haritasında daha öğretici ve düzenli.	1	Zorunlu	routing, middleware, controllers, ProblemDetails
REST	API stili	Kaynakları HTTP fiilleri ve durum kodlarıyla sunar.	Project, ReviewRun ve TestCase kaynaklarını dışa açar.	Basit frontend ve Swagger ile kolay tüketim.	Api	GraphQL, gRPC	Kapsam küçük; REST daha görünür ve öğretici.	1	Zorunlu	resource design, idempotency, status codes
Built-in DI	Mimari	Bağımlılıkları container üzerinden çözer.	Interface-implementation eşlemesi ve yaşam döngüsü.	Test doubles ve dependency inversion sağlar.	Api composition root	Autofac	Ek paket gerektirmez.	1	Zorunlu	scoped/transient/singleton, composition root
DTO + DataAnnotations	Doğrulama	API sözleşmesini ve temel alan kurallarını tanımlar.	Request/response modellerini domain'den ayırır.	Başlangıç kapsamı için yeterli; ek dependency yok.	Api contracts	FluentValidation	Kurallar basitken framework içi çözüm yeterli; çapraz alan kuralları büyürse FluentValidation değerlendirilebilir.	2	Zorunlu	model binding, validation, boundary models
System.Text.Json	Serialization	JSON serileştirme kütüphanesi.	API ve LLM structured output modellerini işler.	.NET içinde yerleşik ve hızlıdır.	Api + Infrastructure	Newtonsoft.Json	Ek paket gerektirmez; enum converter kontrol edilebilir.	2	Zorunlu	JSON options, enum strings, strict deserialization
PostgreSQL	Database	İlişkisel açık kaynak veritabanı.	Project, ReviewRun, Finding, TestCase, AIModelRun ve ToolExecution saklar.	İlişkiler, constraint ve geçmiş sorguları için uygundur.	Persistence	SQL Server, SQLite	Ücretsiz, güçlü ve portföyde gerçek DB deneyimi sağlar.	2	Zorunlu	PK/FK, index, transaction, relational modeling
Npgsql	Driver	.NET ile PostgreSQL arasındaki ADO.NET sağlayıcısı.	EF Core'un PostgreSQL'e bağlanmasını sağlar.	Resmi ekosistem standardıdır.	Infrastructure	Raw ADO.NET driver	EF provider entegrasyonu olgun ve tip uyumludur.	2	Zorunlu	connection string, provider, pooling
EF Core 10	ORM	LINQ ile ilişkisel veri erişimi sağlar.	Mapping, sorgu, change tracking ve transaction.	Boilerplate azalır, yine de SQL davranışını görünür kılar.	Infrastructure.Persistence	Dapper, raw SQL	CRUD ve ilişkiler baskın; Dapper gereksiz manuel iş çıkarır.	2	Zorunlu	DbContext, tracking, LINQ translation, transactions
EF Core Migrations	Schema	Model değişikliklerini sürümlü DB adımlarına çevirir.	Local Windows PostgreSQL şemasını kurar/günceller.	Şema geçmişini Git'te görünür yapar.	Infrastructure.Migrations	Manual SQL	Tekrarlanabilir ve EF modeliyle uyumlu.	2	Zorunlu	migration lifecycle, snapshot, rollback
HttpClientFactory	HTTP	Yönetilen HttpClient üretimi ve policy noktası.	Gemini REST çağrıları, timeout ve header yönetimi.	Socket tükenmesini önler; test edilebilir handler sağlar.	Infrastructure.LLM	new HttpClient, üçüncü parti SDK	C# için resmi Google GenAI SDK listesinde C# yok; REST sözleşmesi net ve bağımlılığı az.	4	Zorunlu	typed client, handlers, timeout, cancellation
Gemini API	LLM	Kod bağlamından yapılandırılmış sonuç üreten model servisi.	Review ve test önerisi üretir.	Ücretsiz kota hedefi ve güçlü structured output kabiliyeti.	GeminiProvider	OpenAI, local model	İstenen sağlayıcı; abstraction sayesinde değiştirilebilir.	4	Zorunlu	LLM limits, schema-guided output, rate limits
ILLMProvider	Abstraction	Application'ın sağlayıcıdan bağımsız portudur.	Prompt/context alır, typed sonuç döndürür.	Gemini detaylarını core iş akışından ayırır.	Application	Doğrudan Gemini client	Dependency inversion ve test kolaylığı.	4	Zorunlu	ports/adapters, interface segregation
Structured Output	AI	LLM cevabını şemaya bağlı JSON olarak ister.	Finding ve TestCase adaylarını parse eder.	Serbest metinden daha doğrulanabilir ve saklanabilir.	Agents + LLM	Metin parsing	Hata yüzeyi küçükür; yine de doğrulama zorunlu.	4	Zorunlu	JSON schema, validation pipeline, fail closed
Prompt Engineering	AI	Model talimatlarının açık ve ölçülebilir tasarımı.	Rol, kapsam, çıktı ve yasakları belirler.	Tekrarlanabilir davranış sağlar.	Prompt templates	Ad-hoc strings	Versiyonlama ve eval mümkün olur.	4	Zorunlu	instruction hierarchy, constraints, versioning
Context Engineering	AI	Modele hangi kanıtın hangi bütçeyle verileceğini tasarlar.	Seçilmiş dosya parçaları ve satır metadata'sı üretir.	Maliyet, doğruluk ve güvenliği dengeler.	Scanner + ContextBuilder	Tüm repo'yu gönderme	Kör yükleme token ve secret riskidir.	3	Zorunlu	chunking, budgets, relevance, grounding
Tool Calling	Agent	Modelin tanımlı işlev çağrısı istemesidir.	Gerekli dosyayı okuma/test talebini typed hale getirir.	OS erişimini politikaya bağlar.	Agents + Tools	Modelin shell erişimi	En az yetki ve audit sağlar.	5	Zorunlu	function contracts, authorization, result envelopes
MCP C# SDK	Protokol	Tool ve context sunumunu standartlaştıran resmi C# SDK.	Çalışan safe tool servislerini 1-2 MCP tool olarak expose eder.	Tekrar kullanılabilir istemci entegrasyonu.	Infrastructure.Mcp	Özel JSON-RPC	Resmi Tier 1 C# SDK; ancak tool katmanından sonra eklenir.	6	Opsiyonel-milest one	protocol, transport, capability, adapter
xUnit	Testing	.NET test frameworkü.	Unit, integration ve security testlerini çalıştırır.	Basit, yaygın ve dotnet test ile uyumlu.	tests	NUnit, MSTest	Ekosistem ve örnek desteği güçlü.	1	Zorunlu	facts, theories, fixtures, test isolation

Project	Category	Problem	Context	Goal	Approach	Tools	Outcome	Impact	Value	Notes
WebApplicationFactory	API testing	API'yi test hostunda ayağa kaldırır.	Endpoint, middleware ve DI entegrasyonunu doğrular.	Gerçek HTTP pipeline'ını internetsiz test eder.	IntegrationTests	Manual server	Hızlı ve tekrarlanabilir.	2	Zorunlu	test host, DI override, HTTP assertions
FakeLLMProvider	Test double	Önceden belirlenmiş typed cevap döndürür.	Review akışını Gemini kotası olmadan test eder.	Deterministik, hızlı ve offline test.	Unit + Integration	Mock framework, real API	Davranış odaklı fake daha okunabilir; gerçek API yalnızca manuel smoke test.	4	Zorunlu	fakes, determinism, contract testing
Microsoft.Extensions.Logging	Observability	Structured log abstraction.	Run id, duration, tool ve hata alanlarını kaydeder.	Yerleşik, DI uyumlu ve provider bağımsız.	Tüm katmanlar	Serilog	İlk sürümde ek dependency yok; gerekirse sonra sink eklenir.	2	Zorunlu	structured events, scopes, redaction
OpenAPI	API docs	Makine okunabilir API sözleşmesi.	Endpointleri keşfetme ve manuel deneme.	Frontend ve reviewer için şeffaflık.	Api	Elle dokümantasyon	ASP.NET Core ile yerleşik üretim.	2	Zorunlu	contract documentation, schemas
HTML/CSS/JS	Frontend	Frameworksüz basit web arayüzü.	Project, scan, findings, filters ve test suggestions ekranları.	Backend odağını korur.	Api/wwwroot	React, Blazor	Tek kişinin 28 günlük kapsamına uygun.	7	Zorunlu	fetch, DOM, progressive UI
Git + GitHub	Version control	Değişiklik geçmişi ve portföy yayınlama.	Küçük commitler, issues ve dokümantasyon.	Kanıtlanabilir gelişim süreci.	Repository	Diğer VCS	Staj değerlendirmesinde standart.	0-7	Zorunlu	branching, commits, reviewable history
GitHub Actions	CI	Push/PR üzerinde otomatik doğrulama.	restore, build ve offline test çalıştırır.	Regresyonu erken yakalar.	.github/workflows	Azure DevOps	GitHub ile ücretsiz ve doğal entegrasyon.	7	Zorunlu	CI pipeline, caching, artifacts

7. Neden C# / ASP.NET Core?

.NET 10, 29 Eylül 2026 itibarıyla aktif LTS sürümdür ve Kasım 2028'e kadar desteklenir. EF Core 10 da .NET 10 ile eşleşen LTS sürümdür. Yeni bir portföy projesinde .NET 8 yerine .NET 10 seçmek, destek ömrü ve güncel API deneyimi sağlar. Öğrenme maliyeti açısından controller tabanlı Web API seçilir: endpoint sayısı büyürken route, authorization/filter ve response kontratlarını sınıf bazında düzenli gösterir. Minimal API mümkündür ama bu projede modüler sınırları öğrencinin görmesi için controller daha açıklayıcıdır.

Gemini C# kararı

Google'ın resmi GenAI SDK sayfası C# istemcisi listelemediği için, provider typed HttpClient ile REST üzerinden tasarlanır. Bu bir eksik değil; ILLMProvider arkasında açık bir adapter yazmak HTTP, cancellation, retry ve serialization öğrenimini güçlendirir.

8. Modüler Mimari

Önerilen yapı dört production projesi ve iki test projesidir. Daha fazla proje, ilk sürümde sınırları öğretmekten çok solution yönetimi yükü yaratır.

```
CodeGuardAI.sln
src/CodeGuardAI.Api - composition root, controllers, contracts, wwwroot
src/CodeGuardAI.Application - use-case'ler, agentlar, workflow'lar, portlar
src/CodeGuardAI.Domain - entity, enum, invariant
src/CodeGuardAI.Infrastructure - EF Core, Npgsql, scanner, Gemini, tools, MCP
tests/CodeGuardAI.UnitTests - saf mantık ve security unit testleri
tests/CodeGuardAI.IntegrationTests - API hostu ve opt-in local DB testleri
docs - ADR, diyagram, setup, security ve evaluation
```

Modül	Yer	İşlev	Komponent	Doğrudan Komponent	Input	Output	Teknoloji
API Layer	CodeGuardAI.Api	HTTP, DTO doğrulama, status code, ProblemDetails	Application servisleri	DbContext/Gemini/OS	Request DTO	Response DTO	ASP.NET Core Controllers
Application Layer	CodeGuardAI.Application	Use-case orchestration ve port interface'leri	Domain + interface'ler	Npgsql/HttpClient somutları	Command/query	Result/DTO	C# services, DI abstractions
Domain Layer	CodeGuardAI.Domain	Entity, enum ve temel invariant	Kendi içi	API, EF, Gemini	Domain method args	State/domain errors	Plain C#
Persistence	Infrastructure.Persistence	DbContext, mapping, migrations, query implementasyonu	Application ports + Domain	Controller	Query/specification	Entities/results	EF Core + Npgsql
Repository Scanner	Infrastructure.Repositories	Allowlist ve güvenli dosya keşfi	Application port	LLM	Root + limits	ScanManifest	System.IO
Context Builder	Application.Context	Scan manifestten bütçeli, satır numaralı context	Scanner output	Filesystem doğrudan	ScanManifest	RepositoryContext	Pure C#
LLM Provider	Infrastructure.LLM	Gemini HTTP detayları ve JSON parse	ILLMProvider	DbContext	LLMRequest	LLMResponse<T>	IHttpClientFactory + System.Text.Json
Review Agent	Application.Agents	Review prompt ve finding üretimi	ILLMProvider + context	Gemini somutu/OS	ReviewAgentInput	ReviewAgentResult	C# service
Test Agent	Application.Agents	Finding tabanlı test senaryosu üretimi	ILLMProvider + safe tools	Kod yazma/commit	TestAgentInput	TestGenerationResult	C# service
Tool Layer	Application.Tools + Infrastructure.Tools	Typed güvenli araç sözleşmeleri ve uygulamaları	Scanner/File/Process	Arbitrary shell	Tool input	ToolResult<T>	System.IO/Process
MCP Adapter	Infrastructure.Mcp	Safe tool servislerini protokole map eder	Tool interfaces	Domain DB doğrudan	MCP args	MCP content/result	ModelContextProtocol SDK
Orchestrator	Application.Workflows	Sınırlı adımlı deterministic review/test akışı	Agents, ports, persistence	Kendi kendine loop	Use-case request	Workflow result	C# async workflow
Frontend	Api/wwwroot	İnsan etkileşimi ve sonuç görünümü	REST API	DbContext/Gemini	User actions	HTML UI	HTML/CSS/JS

9. Backend Mimarisi ve Bağımlılık Yönü

CodeGuardAI.Api

HTTP use-case çağırısı

CodeGuardAI.Application

Domain modeli ve kuralları

CodeGuardAI.Domain

Infrastructure, Application tarafından tanımlanan ILLMProvider, IRepositoryScanner ve tool portlarını uygular. Api composition root, bu implementasyonları DI container'a kaydeder. Böylece Application somut Npgsql, Gemini veya MCP paketini bilmez. Bu Dependency Inversion Principle'dır: üst seviye politika ayrıntıya değil, abstraction'a bağlıdır.

Generic repository kararı

EF Core DbContext zaten aggregate koleksiyonu, change tracking ve Unit of Work davranışı sunar. Her entity için IRepository<T> eklemek LINQ yeteneklerini gizleyip anlamsız boilerplate üretir. Bu projede application'a özel query/service portları kullanılır; controller doğrudan DbContext görmez.

10. Domain Tasarımı

Project	Guid Id; Name; RepositoryPath; NormalizedRootPath; CreatedAtUtc; UpdatedAtUtc	Name required; path unique; root erişilebilir	1-N ReviewRuns	RepositoryPath unique index
ReviewRun	Guid Id; ProjectId; Status; StartedAtUtc; CompletedAtUtc?; ModelName; PromptVersion; ErrorCode?; ScanSummaryJson?	Durum geçişleri kontrollü	N-1 Project; 1-N Findings; 1-N AIModelRuns	ProjectId+CreatedAtUtc index
Finding	Guid Id; ReviewRunId; Severity; Category; FilePath; StartLine; EndLine; Title; Reason; Suggestion; Confidence; CreatedAtUtc	0<=confidence<=1; line aralığı gerçek; relative path	N-1 ReviewRun	ReviewRunId, Severity, Category indexleri
TestCase	Guid Id; ReviewRunId; Name; Type; Target; Scenario; Reason; SuggestedTestCode?; CreatedAtUtc	Öneri kaydı; otomatik source modification yok	N-1 ReviewRun	ReviewRunId+CreatedAtUtc index
AIModelRun	Guid Id; ReviewRunId; Purpose; Provider; ModelName; PromptVersion; DurationMs; InputChars; OutputChars; Status; ErrorType?; CreatedAtUtc	Prompt veya secret değil metadata	N-1 ReviewRun	ReviewRunId+Purpose index
ToolExecution	Guid Id; ReviewRunId?; ToolName; Status; DurationMs; InputSummary; OutputSummary; ErrorType?; CreatedAtUtc	Redacted özet; raw secret yok	N-1 optional ReviewRun	ReviewRunId+ToolName index

11. Database Tasarımı

Project (1)

↓

ReviewRun (N)

↓

Finding (N) + TestCase (N) + AIModelRun (N)

↓

ToolExecution (N, ReviewRun opsiyonel)

Project silme operasyonu MVP API'sinde sunulmaz. İleride eklenirse cascade etkisi açık onay ister. ReviewRun çocukları run'ın ayrılmaz geçmişidir ve cascade silinebilir. Finding.FilePath repository root'a göre relative saklanır; makineye özgü absolute path finding'e kopyalanmaz. Enumlar okunabilirlik için string saklanır; check constraint opsiyoneldir. Tüm zamanlar UTC'dir.

12. EF Core Yapısı

- CodeGuardDbContext yalnız Infrastructure'dadır ve scoped yaşam döngüsüne sahiptir.
- Her entity için IEntityConfiguration<T> ile kolon, uzunluk, index ve ilişki açıkça tanımlanır.
- Okuma sorguları AsNoTracking ve projection kullanır; update akışları tracked entity kullanır.
- SaveChangesAsync ve tüm I/O metotları CancellationToken alır.
- Migration'lar Infrastructure içinde tutulur; local Windows PostgreSQL'e dotnet ef database update ile uygulanır.
- EnableSensitiveDataLogging varsayılan kapalı kalır.

Model ayrımları

Domain Entity iş kurallarını taşır. EF configuration bunun relational karşılığını tarif eder. Request Model dış girdiyi alır; Response Model yalnız paylaşılacak alanları döndürür. DTO katmanlar arası veri taşır. LLM Structured Output Model ise güvenilmeyen model JSON'unun ilk parse hedefidir. Tek class'ı her yerde kullanmak over-posting, secret sızıntısı, migration ile API'nin kilitlenmesi ve hatalı LLM verisinin doğrudan domain'e girmesi riskini doğurur.

13. AI / LLM Mimarisi



14. Structured Output

CodeReviewResult, LLMFinding listesi taşır. JSON enumları string olarak yazılır; case-insensitive parse kolaylığı kullanılabilir ancak bilinmeyen enum fail-closed reddedilir. Required alanlar Title/Reason/Suggestion dahil boş olamaz. Confidence 0-1 aralığında decimal; StartLine ≥ 1 , EndLine $\geq$ StartLine olmalıdır. Parse başarılı olması yeterli değildir: dosya manifestte ve satırlar gerçek dosya aralığında bulunmalıdır.

JSON -> System.Text.Json deserialize -> structural validation -> enum/range validation -> file/line grounding -> de-duplication -> Domain Finding -> EF Core persistence

15. Model ve AI Kuralları

Structured output	response MIME/schema + strict DTO validation	Run Failed/Partial değil; invalid provider response olarak kaydet
Grounding	Yalnız contextteki relative path ve 1-based line range	Finding'i reject et; sayısını metric olarak tut
Context limits	Max files, file bytes, total chars, deterministic priority	Scan/context sonucu limit nedeniyle skipped işaretler
Prompt versioning	review-system.v1 ve test-system.v1; AIModelRun metadata	Versiyon yoksa çağrı yapılmaz
Retry	Yalnız 429/502/503 gibi transient hata; max 2	Retry bitince typed provider error
Timeout	HttpClient + linked CancellationToken	ReviewRun Failed: ProviderTimeout
Secrets	Denylist + User Secrets + redacted logs	Şüpheli dosya skip; key asla loglanmaz
Budget	Karakter tabanlı konservatif budget; token tahmini metadata	Bütçe dolunca deterministic truncation
Hallucination	Unknown file/line quarantine/reject	DB'ye Finding olarak girmez

System Prompt, User Prompt, Context, Tool Result, Response

System Prompt	Agent rolü, güvenlik sınırı, kategori ve çıktı kuralları; versioned template.
User Prompt	Bu review'e özel hedef: proje, analiz modu ve seçilmiş istek.
Repository Context	Scanner ve ContextBuilder'ın ürettiği, dosya/satır kaynaklı kanıt.
Tool Result	Tool'un başarı/error/metadeta içeren güvenilmeyen fakat typed sonucu; model talimatı değildir.
Structured Response	LLM'in şemaya uyması beklenen JSON'u; doğrulanana kadar domain verisi değildir.

16. Agent Mimarisi

İlk sürümde iki agent yeterlidir. Üçüncü Summary Agent eklenmez; özet, deterministic application projection ile üretilebilir. Agent adı, bağımsız yaşayan süreç anlamına gelmez; prompt + input contract + validation davranışı olan uygulama servsidir.

17. Review Agent

Görev	RepositoryContext içinde bug, security, maintainability, performance, reliability ve code quality bulgusu üretmek.
Input	Project metadata, versioned prompt, bounded context, allowed categories.
Output	CodeReviewResult -> grounded Finding listesi.
Kullanabilir	İlk sürümde context hazır gelir; gerekirse IFileReadTool yalnız manifest içindeki dosya için.
Kullanamaz	ITestRunnerTool, shell, write_file, git commit, network dışında provider.
Başarı	JSON geçerli, her path/line gerçek, en fazla configured finding sayısı.
Hata	Typed error; uydurma düzeltme yok; ReviewRun terminal Failed olur.

18. Test Agent

Görev	Completed review bulguları ve ilgili koddan unit/edge/regression/validation test senaryosu önermek.
Input	ReviewRun, selected findings, target context.
Output	TestGenerationResult -> TestCase listesi.
Kullanabilir	IFileReadTool; test çalıştırma yalnız kullanıcı ayrı run isteği verirse orchestrator üzerinden.
Kullanamaz	Production source yazma, git, arbitrary shell.
Başarı	Target grounded; scenario ve reason dolu; duplicate yok.
Hata	Mevcut review/finding bozulmaz; AIModelRun error metadeta saklanır.

19. Orchestrator

Orchestrator agentların birbirine mesaj gönderdiği serbest bir ağ değildir. Sıralı application workflow'dur. Review akışı en fazla bir scan, bir context build ve bir model call içerir; bounded retry provider içindedir. Test önerisi ayrı workflow'dur. Her adım CancellationToken alır, terminal state üretir ve aynı run'ı iki kez başlatmayı engeller.

20. Tool Layer

Tool	Expected Input	Expected Output	Limit	Limit Note
IRepositoryScanTool	ProjectId/root + ScanOptions	ToolResult<ScanManifest>	Kayıtlı normalized root	file count/bytes; reparse reject
IFileReadTool	ProjectId + relative path	ToolResult<FileContent>	Manifest allowlist	root confinement; UTF-8; max bytes
ITestRunnerTool	ProjectId + allowlisted test target	ToolResult<TestRunSummary>	Sabit dotnet test operasyonu	timeout; output cap; process-tree kill

Command injection savunması

ITestRunnerTool string command kabul etmez. ProcessStartInfo.UseShellExecute=false kullanır, executable ve 'test' sabittir, kullanıcı girdisi ArgumentList'e tek argüman olarak eklenir, target canonical root altında doğrulanır. cmd.exe veya PowerShell çağrılmaz.

21. MCP Mimarisi

Tool calling, modelin uygulama tarafından tanımlanmış bir işlevi çağırma talebidir. MCP ise tool/resource/prompt keşfi ve çağrısını istemci-sunucu arasında standartlaştıran protokoldür. MCP Server bu sözleşmeleri transport üzerinden sunar. MCP tek başına sandbox değildir; authorization, path ve process güvenliği alttaki tool servisinde kalır.

MCP Client / Agent Host

↓

MCP C# SDK Adapter

↓

IFileReadTool / ITestRunnerTool

↓

Safe Implementations

↓

Filesystem / dotnet test

İlk sürüm read_file ve run_tests ile sınırlıdır. Official MCP C# SDK Tier 1 olarak belgelenmiştir. Local kullanım için stdio en sade seçenektir; aynı ASP.NET host içinde HTTP transport ancak gerçek bir remote client ihtiyacı doğarsa değerlendirilir. Protocol version pinning yerine SDK'nın uyumluluğu kullanılmalı, fakat paket sürümü lock file/central package yönetimiyle sabitlenmelidir.

22. Güvenlik Kuralları

Problem	Konsept	Özellik
Path traversal	Canonical root + separator-aware containment	../, absolute sibling, mixed separator
Secret exposure	Name/path denylist, no raw logs, bounded context	.env, *.pem, appsettings secret fixture
Prompt injection	Repository content DATA delimiter içinde; tool authorization modelden bağımsız	'ignore instructions' içeren source fixture
Unsafe file access	Manifest membership + extension allowlist + reparse reject	symlink/reparse ve binary
Command injection	No shell, fixed verb, ArgumentList, allowlisted target	; & metacharacters and outside path
Huge context	Per-file/total/file-count caps	oversized repo/file
Malformed JSON	Strict parse + schema + typed errors	missing, extra, wrong enum/type
Hallucinated line	Manifest and line-count grounding	unknown path/out-of-range
API key exposure	User Secrets/env; redacted logging	log capture and git grep

23. Ana Özellikler

- Local project kaydı ve history
- Güvenli repository scan manifesti
- Bütçeli ve satır kaynaklı context
- Gemini structured code review
- Grounded findings ve filtreleme
- Review tabanlı test önerileri
- Explicit safe dotnet test run
- AI/tool metadata ve structured logs
- İki safe tool için MCP adapter
- Basit frameworksüz web UI

24. API Haritası

GET	/health	Liveness ve temel durum	-	HealthResponse	Health	Yok	Hayır	200/503	Bağımlılık ayırımı; secret yok
POST	/api/projects	Yerel repo kaydı	CreateProjectRequest	ProjectResponse	ProjectService	Project INSERT	Hayır	201/400/409/422	Ad, absolute path, erişim, unique normal root
GET	/api/projects	Proje listesi	Query: page, pageSize	Paged<ProjectSummary>	ProjectQueryService	Read-only	Hayır	200/400	Page size üst sınırı
GET	/api/projects/{id}	Proje ayrıntısı	Guid route	ProjectDetailResponse	ProjectQueryService	Read-only	Hayır	200/404	Guid ve existence
POST	/api/projects/{id}/scan	Güvenli tarama	ScanProjectRequest? limits	ScanResultResponse	RepositoryScanService	İsteğe bağlı summary update	Hayır	200/404/409/422	Root, allowlist, size, reparse point
POST	/api/projects/{id}/reviews	Review workflow başlat	CreateReviewRequest	ReviewRunResponse	ReviewOrchestrator	ReviewRun/AIModelRun/Finding INSERT	Evet	201/404/409/422/502/504	Scan/context limits, provider availability
GET	/api/reviews/{id}	Review ve findings	Guid route	ReviewDetailResponse	ReviewQueryService	Read-only	Hayır	200/404	AsNoTracking, projection
GET	/api/projects/{id}/reviews	Review geçmişi	Paging/filter	Paged<ReviewSummary>	ReviewQueryService	Read-only	Hayır	200/404/400	Status/date filters
POST	/api/reviews/{id}/tests	Test önerileri üret	GenerateTestsRequest	TestGenerationResponse	TestOrchestrator	AIModelRun/TestCase INSERT	Evet	201/404/409/422/502/504	Completed review, bounded findings/cor
POST	/api/reviews/{id}/test-runs	İzinli dotnet test çalıştır	RunTestsRequest	ToolExecutionResponse	SafeTestRunner	ToolExecution INSERT	Hayır	202/400/404/409/504	Allowlisted project/test target; no raw co

25. Testing Stratejisi

Test Türü	Test Senaryoları	Test Ortamı / Test Aracıları
Unit	Domain invariant, mapper, budget, grounding, workflow decisions	I/O yok; hızlı ve deterministik
Integration	API host, DI, middleware, fake LLM ile review/test flow	WebApplicationFactory; gerçek HTTP pipeline
Database Integration	Npgsql mapping, migration, FK/index ve query	Local PostgreSQL opt-in Category=Database; EF InMemory relational davranış kanıtı değildir
LLM Provider Contract	HTTP request, timeout, retry, JSON parse	Fake HttpMessageHandler; gerçek Gemini yok
Scanner Security	Traversal, reparsing, secret, huge file, encoding	Temporary directory fixtures
Tool Security	Raw args rejection, timeout, output cap	Fake process abstraction veya kontrollü fixture
Structured Output	Valid/malformed/unknown enum/file/line	Fixture JSON; fail-closed
Manual Smoke	Gerçek Gemini ile tek küçük fixture	Normal suite dışında, açık opt-in ve kota bilinci

Mock yalnız dış nondeterminism'i izole etmek için kullanılır: Gemini HTTP, zaman, filesystem veya process sınırı. Domain nesnelerini aşırı mocklamak testleri implementasyona bağlar. FakeLLMProvider semantik fixture döndürdüğü için mock çağrı zincirinden daha okunabilir olur.

26. LLM Evaluation Stratejisi

Eval fixture'ları küçük ve bilinen kusurludur: null dereference, eksik input validation, path traversal, IDisposable leak ve temiz kontrol örneği. Exact title/reason string eşitliği aranmaz. Ölçütler: expected category bulundu mu, expected file eşleşti mi, line gerçek mi, suggestion boş değil mi, JSON geçerli mi, false positive control'da kritik bulgu üretildi mi? Sonuçlar prompt version ve model name ile raporlanır.

27. Logging / Observability

Log eventleri mesaj metnine gömülü değil, alanlı olmalıdır: ReviewRunId, ProjectId, Model, PromptVersion, DurationMs, FindingCount, RejectedFindingCount, ToolName, ToolDurationMs, ErrorType. Logging scope her workflow'a correlation sağlar. API key, password, connection string, raw source, raw prompt ve raw model response varsayılan olarak loglanmaz. AIModelRun ve ToolExecution veritabanında audit metadata taşır; log ile aynı şey değildir.

28. Git / GitHub Stratejisi

- Her günün acceptance criteria'sı geçmeden commit oluşturma.
- Küçük, açıklayıcı commit: feat(scanner): enforce repository root boundary gibi.
- main her zaman build/test geçer; kısa feature branch opsiyoneldir.
- ADR ile önemli kararları commit'e bağla.
- Generated secrets, local paths ve test outputs için .gitignore kullan; örnek config secret içermez.

29. GitHub Actions CI

CI: checkout -> setup-dotnet 10.x -> restore -> build --no-restore -> test --no-build. Unit/API/FakeLLM testleri internet, Gemini key veya PostgreSQL istemez. Local PostgreSQL gerektiren testler Category=Database ile dışlanır ve geliştirici makinesinde ayrıca çalıştırılır. Bu bilinçli trade-off, container yaşağını korur; CI'nın relational doğrulaması sınırlı kaldığı README'de dürüstçe belirtilir.

30. Geliştirme Fazları

Faz 0 - Project Definition & Architecture	Kapsam, tehdit modeli, ADR ve dependency sınırları	Koddan önce aynı sözlüğü kurmak	Yok	Git, Markdown, diagrams	docs/ ve solution skeleton planı	Onaylı scope + ADR	Docker/Python yok; modüler monolit	Overplanning	trade-off, boundaries
Faz 1 - ASP.NET Core Foundation	Solution, API, DI, error model, OpenAPI	Sonraki modüller için çalışır iskelet	Faz 0	.NET 10, ASP.NET Core, xUnit	Api/Application/Domain/Infrastructure	Health ve test hostu	build/test yeşil	Katman ihlali	HTTP pipeline, DI
Faz 2 - PostgreSQL + EF Core	İlişkisel model ve Project/Review persistence	AI verisi öncesi güvenilir kayıt modeli	Faz 1	EF Core 10, Npgsql, local PostgreSQL	DbContext/config/migrations	CRUD + geçmiş sorguları	Migration local DB'de uygulanır	Şema drift	mapping, migrations
Faz 3 - Scanner & Context	Güvenli repo tarama ve bütçeli context	LLM'e temiz kanıt sağlamak	Faz 2	System.IO, options	Scanner, reader, context builder	ScanManifest + RepositoryContext	Traversal/reparse/size testleri	Secret sızıntısı	filesystem security, context budgets
Faz 4 - Gemini Review	Provider abstraction, typed output, grounding	Core ürün değerini kurmak	Faz 3	HttpClientFactory, Gemini REST, JSON	ILLMProvider, GeminiProvider, ReviewAgent	Persisted findings	Fake tests + manuel smoke	Malformed JSON/rate limits	DIP, structured AI
Faz 5 - Test Agent & Tools	Test önerisi ve güvenli test geliştirme	Review değerini eyleme dönüştürmek	Faz 4	xUnit, ProcessStartInfo	TestAgent, tool contracts	Test cases + bounded runner	No arbitrary command; timeout	Command injection	tool contracts, process safety
Faz 6 - MCP & Orchestration	Çalışan tools üzerine küçük MCP adapter ve deterministik workflow	Standart entegrasyonu doğru sırada eklemek	Faz 5	Official MCP C# SDK	MCP server adapter, orchestrators	read_file/run_tests exposure	MCP contract tests	Protocol churn	adapter pattern, max steps
Faz 7 - UI, Eval, CI, Docs	Kullanılabilir demo ve kanıtlanabilir kalite	Ürünü portföye hazır hale getirmek	Faz 6	HTML/CSS/JS, Actions	wwwroot, eval fixtures, workflow	Demo-ready repository	offline CI green	Polish scope creep	delivery, evaluation, CI

31. 28 Günlük Ayrıntılı Geliştirme Planı

Çalışma ritmi

Her gün önce bir önceki günün acceptance criteria'sı doğrulanır. Bugünkü görev küçük bir vertical slice veya tek bir güvenlik/mimari sınırdır. Gün bitiminde build/test sonucu, öğrenilen kavram ve teknik borç kaydı yazılır. Bir gün tamamlanmadan sonraki güne geçilmez.

Aşağıdaki 28 günlük bölümde aynı şablon kullanılır. Dosya yolları hedef yapıyı öğretir; gerçek repository incelendikten sonra isimler küçük farklılık gösterebilir. Codex, çelişki görürse sessizce değiştirmek yerine raporlamalıdır.

Özellik	Kodun kullanılması
Günlük uygulama planı	Hedef, mimarideki yer, dosyalar, yapılacaklar, yasaklar ve acceptance criteria birlikte okunur.
Codex Komutu	Günün sonundaki kısa komut, o günün planını genel kurallara bağlı olarak uygular.
Genel çıktı kontrolü	Tek genel şablondaki dokuz parçalı teslim özeti ve test sonucu görülmeden gün tamamlandı sayılmaz.
Öğrenme notu	Günün sonunda teknoloji adını değil, çözdüğü problemi kendi cümleleriyle açıklarsın.

İlerleme kuralı

Bir gün bloklandıysa sonraki günün özelliğini erkenden eklemek yerine blokajı, denenenleri ve gerekli kullanıcı kararını kaydet. Plan sırası teknik bağımlılık sırasındır.

31.1 Codex Genel Uygulama Kuralları

Mimari yaklaşım

CodeGuard AI, modüler monolith düzeninde ve Clean Architecture prensipleriyle geliştirilir. Üretim projeleri Domain, Application, Infrastructure ve API; test projeleri UnitTests ve IntegrationTests olarak korunur.

Domain	Entity, value object, enum ve iş invariant'ları. Başka katmana, framework'e veya I/O ayrıntısına bağımlı değildir.
Application	Use case, port/interface, DTO ve workflow kuralları. Domain'e bağımlıdır; Infrastructure implementasyonlarını bilmez.
Infrastructure	EF Core/PostgreSQL, Gemini, filesystem, process ve MCP adaptörleri. Application portlarını uygular; dış sistem ayrıntılarını içerir.
API	Controller/endpoint, middleware ve composition root. Bağımlılıkları birleştirir; iş kuralını controller içine taşımaz.

Bağımlılık yönü API → Application → Domain ve Infrastructure → Application + Domain şeklindedir. Modüller sözleşmeler üzerinden haberleşir; katman sınırları günlük hız uğruna delinmez.

Ortak çalışma kuralları

- Önce mevcut repository'yi ve yalnız bugünkü görevle ilgili dosyaları incele; mimariyi ve mevcut davranışı anlamadan değişiklik yapma.
- Yalnız bugünkü adımın kapsamını uygula. Sonraki fazın özelliğini erkenden ekleme; acceptance criteria tamamlanınca dur ve sonraki güne geçme.
- Çalışan kodu gereksiz yere yeniden yazma. Değişiklikleri küçük, geri alınabilir ve mevcut public contract'larla uyumlu tut.
- Modüler Clean Architecture sınırlarını, bağımlılık yönünü, cancellation ve açık hata davranışını koru.
- Docker veya başka bir container teknolojisi ekleme. Python/FastAPI kullanma; backend C# / ASP.NET Core olarak kalmalı ve PostgreSQL local Windows üzerinden kullanılmalı.
- Secret, API key, password veya connection string'i kaynak koda ya da repository'ye yazma. Gemini API key'i environment variable veya User Secrets ile sağla.
- Gereksiz NuGet paketi ekleme. Yeni bağımlılığın çözdüğü problemi ve mevcut araçlarla neden çözemediğini açıklayabil.
- Değişen dosyaları ve her değişikliğin nedenini belirt; gerekli testleri ekle veya güncelle; doğrulama komutlarını ve gerçek sonuçlarını kaydet.
- Mevcut repository ile günlük talimat çelişirse sessizce mimariyi değiştirme; çelişkiyi, etkisini ve gerekli kararı açıkça bildir.
- Günün teknolojik odağını, trade-off'unu ve başarısızlık davranışını sade dille açıklayabilecek kadar öğrenme notu çıkar.
- İşlem sırasında gereksiz ara rapor üretme; yalnız karar gerektiren blokajı ve tamamlanan sonucun kısa özetini bildir.
- Her gün yalnızca o günkü görev için gerekli repository dosyalarını incele; repository'nin tamamını gereksiz yere tekrar analiz etme. İlgili dosyaları belirle, görevi uygula, gerekli test/doğrulamaları yap ve acceptance criteria sağlandığında dur. Sonraki günün kapsamına geçme.

31.2 Configuration, Options ve Hardcode Politikası

Hardcode yasağı

Ortamdan, sağlayıcıdan, güvenlik politikasından veya operasyonel ihtiyaçtan dolayı değişebilecek hiçbir değer uygulama akışına gömülmez. Configuration, strongly typed Options, environment variables ve uygun abstraction'lar kullanılır.

Özellik	Uygulanma Durumu
Secret ve bağlantılar	API key, password ve connection string: environment variable / User Secrets; repository'de gerçek değer yok.
LLM davranışı	Model adı, endpoint, timeout, retry, prompt version ve çıktı limitleri: strongly typed Options ve sağlayıcı abstraction'ı.
Repository güvenliği	İzinli uzantılar, reddedilen dizinler, dosya/context boyutları ve traversal politikası: merkezi policy/options nesneleri.
Tool ve test sınırları	Process timeout, output cap, izinli komut/argüman ve test-runner limitleri: doğrulanan Options ile güvenli adaptörler.
Ürün limitleri	Maksimum finding, pagination ve benzeri operasyonel limitler: configuration + validation; magic number yok.

Domain invariant'ları, enum değerleri, protokolün zorunlu sabitleri ve değişmesi beklenmeyen matematiksel sabitler hardcode sayılmaz; adlandırılmış ve anlamı görünür sabitler olarak tutulabilir. Configuration değerleri başlangıçta validate edilir ve eksik/uygunsuz durumda fail-fast davranır.

31.3 Codex Çıktı Kontrol Şablonu

Sıra	Kontrol Kriteri
1. Yapılanlar	Yalnız bugünkü kapsam; sonraki fazdan özellik yok.
2. Değişen dosyalar	Her dosya yolu eksiksiz listelenmiş.
3. Dosya amacı	Her değişikliğin mimarideki görevi açıklanmış.
4. Mimari kararlar	Trade-off ve bağımlılık yönü belirtilmiş.
5. Testler	Eklenen/güncellenen testler ve gerçek sonuçları mevcut.
6. Doğrulama	Komutlar kopyalanabilir; beklenen ve gerçekleşen sonuç yazılı.
7. Öğrenme	Bugünün kavramları sade dille açıklanmış.
8. Risk / teknik borç	Kalan risk gizlenmeden kaydedilmiş.
9. Hazırlık	Acceptance criteria sağlandı; sonraki güne hazır / hazır değil gerekçeli.

Tamamlama koşulu

Bu şablon günlük planlarda tekrarlanmaz. Her günün teslimi bu tek şablonla kontrol edilir; test/doğrulama kanıtı ve acceptance criteria tamamlanmadan sonraki güne geçilmez.

GÜN 1 / 28

Kapsam ve başarı ölçütleri

Faz 0
UYGULAMA PLANI

BU FAZIN HEDEFİ	Kapsamı ve kararları koddan önce sabitlemek.
BU ADIMDA NE YAPIYORUZ?	Ürünün sınırlarını, aktörlerini, in-scope/out-of-scope özelliklerini ve 28 günlük Definition of Done'ı yazılı hale getiriyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Architecture kararlarından önce ortak hedef gerekir; aksi halde sonraki günler farklı ürünler inşa eder.
BU ADIMIN MİMARİDEKİ YERİ	docs/project-scope.md, docs/glossary.md merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	Git, Markdown

YAPILACAKLAR

- Ana demo akışını ve kullanıcı hikayelerini yaz
- Docker, Python, otomatik kod değiştirme ve production müdahalesini out-of-scope kaydet
- Finding alanlarını ve review/test başarı ölçütlerini tanımla
- Ücretsiz/local-first varsayımlarını kaydet
- Risk register'ın ilk sürümünü oluştur

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

docs/project-scope.md, docs/glossary.md

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

İşlev kodu, NuGet paketi veya database oluşturma.

BU ADIM BİTTİĞİNDE

Scope belgesi onaylanabilir ve her özellik bir başarı ölçütüne bağlıdır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: Git, Markdown.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Scope içinde Docker/Python yok
- ☐ Demo flow ve DoD net
- ☐ Riskler sahipli ve azaltma adımı

TEST / DOĞRULAMA

Belgeleri gözden geçir; çelişkili terim ara.

CODEx KOMUTU

Gün 1 - Kapsam ve başarı ölçütleri planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 2 / 28

Mimari ve solution iskeleti

Faz 0 / Faz 1
UYGULAMA PLANI

BU FAZIN HEDEFİ	Kararları derlenebilir solution sınırlarına dönüştürmek.
BU ADIMDA NE YAPIYORUZ?	Dört proje ve iki test projesinden oluşan modular monolith solution kurup dependency direction'ı kilitliyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Mimari sınırlar fiziksel referanslarla erken korunur.
BU ADIMIN MİMARİDEKİ YERİ	CodeGuardAI.sln merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	.NET 10 SDK, C#, Git

YAPILACAKLAR

- .NET 10 solution ve projeleri oluştur
- Project reference yönlerini Api->Application, Infrastructure->Application+Domain, Application->Domain olarak kur
- Nullable ve warnings-as-errors politikasını değerlendirip Directory.Build.props'a yaz
- ADR-001 modular monolith ve ADR-002 controller seçimini ekle
- İlk dotnet build ve dotnet test çalıştır

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

```
CodeGuardAI.sln; src/CodeGuardAI.Api; Application; Domain; Infrastructure; tests/UnitTests; IntegrationTests; docs/adr/
```

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Infrastructure'ı Domain'e sızdırma; feature kodu ekleme.

BU ADIM BİTTİĞİNDE

Solution derlenir, iki test projesi keşfedilir ve yasak dependency yoktur.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: .NET 10 SDK, C#, Git.

KONTROL / ACCEPTANCE CRITERIA

- ☐ dotnet build başarılı
- ☐ dotnet test başarılı
- ☐ Reference graph belgelenmiş

TEST / DOĞRULAMA

```
dotnet restore; dotnet build; dotnet test
```

CODEx KOMUTU

Gün 2 - Mimari ve solution iskeleti planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 3 / 28

API composition root ve health

Faz 1
UYGULAMA PLANI

BU FAZIN HEDEFİ	Çalışan, test edilebilir ASP.NET Core temelini kurmak.
BU ADIMDA NE YAPIYORUZ?	ASP.NET Core Web API, controller routing, DI registration alanı, OpenAPI ve health endpointi kuruyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Çalışan en küçük dikey dilim sonraki günlerin güvenlik ağıdır.
BU ADIMIN MİMARİDEKİ YERİ	Api/Program.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	ASP.NET Core, Controllers, OpenAPI, WebApplicationFactory

YAPILACAKLAR

1. Controller tabanlı API template'ini temiz kur
2. MapControllers, ProblemDetails ve development OpenAPI ekle
3. GET /health endpointini typed response ile oluştur
4. WebApplicationFactory için Program partial erişimini sağla
5. Health endpoint integration testini yaz

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Api/Program.cs; Api/Controllers/HealthController.cs; Api/Contracts/HealthResponse.cs;
IntegrationTests/HealthEndpointTests.cs

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

DB veya Gemini bağlantısı ekleme; health içinde secret döndürme.

BU ADIM BİTTİĞİNDE

API dotnet run ile açılır; /health 200 ve test hostunda doğrulanır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümleleriyle anlat: ASP.NET Core, Controllers, OpenAPI, WebApplicationFactory.

KONTROL / ACCEPTANCE CRITERIA

- ☐ OpenAPI development'ta erişilir
- ☐ /health 200 döner
- ☐ Integration test internetsiz geçer

TEST / DOĞRULAMA

dotnet run --project src/CodeGuardAI.Api; dotnet test

CODEX KOMUTU

Gün 3 - API composition root ve health planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 4 / 28

HTTP sözleşmesi, validation ve hata modeli

Faz 1
UYGULAMA PLANI

BU FAZIN HEDEFİ	Çalışan, test edilebilir ASP.NET Core temelini kurmak.
BU ADIMDA NE YAPIYORUZ?	DTO, DataAnnotations, ProblemDetails ve application result yaklaşımını tanımlıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Tutarlı API davranışı olmadan feature endpointleri farklı hata biçimleri üretir.
BU ADIMIN MİMARİDEKİ YERİ	Api/Contracts/Common merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	C#, ASP.NET Core, DataAnnotations, ProblemDetails

YAPILACAKLAR

1. Request/response/domain ayrımını örnek Project contract ile kur
2. DataAnnotations için sınırları belirle
3. Validation hatalarını RFC 9457 ProblemDetails biçiminde standardize et
4. NotFound, Conflict ve validation error mapping tasarla
5. Result türünün unit testlerini yaz

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

```
Api/Contracts/Common; Api/Errors; Application/Common/Result.cs; UnitTests/ResultTests.cs
```

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Domain entity'yi doğrudan response olarak döndürme; exception ile normal akış yönetme.

BU ADIM BİTTİĞİNDE

Örnek invalid request deterministic 400 problem response verir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: C#, ASP.NET Core, DataAnnotations, ProblemDetails.

KONTROL / ACCEPTANCE CRITERIA

- ☐ DTO-domain ayrımı görünür
- ☐ Error code ve trace id var
- ☐ Validation testi geçer

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~Result
```

CODEx KOMUTU

Gün 4 - HTTP sözleşmesi, validation ve hata modeli planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 5 / 28

Configuration ve secret güvenliği

Faz 1
UYGULAMA PLANI

BU FAZIN HEDEFİ	Çalışan, test edilebilir ASP.NET Core temelini kurmak.
BU ADIMDA NE YAPIYORUZ?	Options pattern, environment override ve User Secrets kullanımını planlayıp uygulama sınırlarını kuruyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Secret yönetimi ilk gerçek external dependency'den önce çözülmelidir.
BU ADIMIN MİMARİDEKİ YERİ	Api/appsettings.json merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	Options pattern, User Secrets, environment variables

YAPILACAKLAR

- Database ve Gemini ayarları için strongly typed options tanımla
- Startup validation ekle ancak secret'ı loglama
- GeminiApiKey değerini appsettings dosyalarına koyma
- Development için dotnet user-secrets komutlarını belgeye ekle
- Config precedence ve hata davranışını test et

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Api/appsettings.json; Api/appsettings.Development.json; Application/Options; docs/setup/configuration.md

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Gerçek connection string/API key commit etme; config dump loglama.

BU ADIM BİTTİĞİNDE

Eksik zorunlu ayar güvenli ve anlaşılır biçimde raporlanır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: Options pattern, User Secrets, environment variables.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Repository'de secret yok
- ☐ Options validate edilir
- ☐ Setup komutları belgeli

TEST / DOĞRULAMA

dotnet user-secrets list çıktısını belgeye kopyalamadan manuel kontrol; dotnet test

CODEx KOMUTU

Gün 5 - Configuration ve secret güvenliği planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 6 / 28

Domain entity ve enumlar

Faz 2
UYGULAMA PLANI

BU FAZIN HEDEFİ	İlişkisel geçmişi güvenilir ve tekrarlanabilir saklamak.
BU ADIMDA NE YAPIYORUZ?	Project, ReviewRun, Finding, TestCase, AIModelRun ve ToolExecution modellerini invariant odaklı kuruyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Veritabanı mappinginden önce iş kuralları framework bağımsız tanımlanır.
BU ADIMIN MİMARİDEKİ YERİ	Domain/Projects merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	C#, nullable reference types, xUnit

YAPILACAKLAR

- Guid kimlik ve UTC timestamp politikasını belirle
- ReviewStatus ve finding/test/tool enumlarını oluştur
- ReviewRun durum geçişlerini methodlarla sınırla
- Finding confidence ve line range invariantlarını uygula
- Entity unit testlerini ekle

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

`Domain/Projects; Domain/Reviews; Domain/Tests; Domain/Observability; UnitTests/Domain`

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Public setter ile her durumu mümkün kılma; EF attribute'larını domain'e doldurma.

BU ADIM BİTTİĞİNDE

Geçersiz state oluşturulamaz, enumlar tek sözlükle kullanılır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: C#, nullable reference types, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Invariant testleri geçer
- ☐ UTC politikası net
- ☐ Domain Infrastructure referansı içermez

TEST / DOĞRULAMA

`dotnet test --filter FullyQualifiedName~Domain`

CODEX KOMUTU

Gün 6 - Domain entity ve enumlar planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 7 / 28

DbContext ve EF mappings

Faz 2
UYGULAMA PLANI

BU FAZIN HEDEFİ	İlişkisel geçmişi güvenilir ve tekrarlanabilir saklamak.
BU ADIMDA NE YAPIYORUZ?	Domain modellerini PostgreSQL tablolarına explicit configuration ile eşliyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Explicit mapping şemayı görünür ve test edilebilir kılar.
BU ADIMIN MİMARİDEKİ YERİ	Infrastructure/Persistence/CodeGuardDbContext.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	EF Core 10, Npgsql, PostgreSQL

YAPILACAKLAR

- DbSet'leri ve IEntityTypeConfiguration sınıflarını ekle
- PK/FK, required, max length, enum string ve precision ayarlarını yap
- Cascade davranışını Project->ReviewRuns cascade, ReviewRun çocukları cascade; keyfi project silmeyi service seviyesinde koru
- Unique Project.NormalizedRootPath indexini ekle
- Npgsql DbContext'i scoped DI ile kaydet

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Infrastructure/Persistence/CodeGuardDbContext.cs; Configurations/*.cs; Api/DependencyInjection.cs

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Generic repository ekleme; sensitive data logging'i açma.

BU ADIM BİTTİĞİNDE

Model metadata beklenen constraint ve ilişkileri içerir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: EF Core 10, Npgsql, PostgreSQL.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Model validation warning yok
- ☐ Index/relationships tanımlı
- ☐ Application Npgsql'a referans vermez

TEST / DOĞRULAMA

dotnet build; dotnet test

CODEX KOMUTU

Gün 7 - DbContext ve EF mappings planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 8 / 28

Migration ve local PostgreSQL kurulumu

Faz 2
UYGULAMA PLANI

BU FAZIN HEDEFİ	İlişkisel geçmişi güvenilir ve tekrarlanabilir saklamak.
BU ADIMDA NE YAPIYORUZ?	İlk migration'ı üretip local Windows PostgreSQL üzerinde uygulama ve geri alma yolunu doğruluyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Migration'ın yalnızca oluşması değil, gerçek DB'de uygulanması gerekir.
BU ADIMIN MİMARİDEKİ YERİ	Infrastructure/Persistence/Migrations/* merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	EF Core CLI, PostgreSQL Windows, Npgsql

YAPILACAKLAR

- Local PostgreSQL database/user için least-privilege adımlarını belge
- InitialCreate migration üret
- SQL scriptini gözden geçir: tablo, FK, index, enum storage
- dotnet ef database update çalıştır
- Test database'i için ayrı connection string ve temizleme yaklaşımı tanımla

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Infrastructure/Persistence/Migrations/*; docs/setup/postgresql-windows.md

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Docker kullanma; postgres superuser credential'ını repo'ya koyma; migration dosyasını elle rastgele düzeltme.

BU ADIM BİTTİĞİNDE

Boş local DB tek komutla güncel şemaya gelir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: EF Core CLI, PostgreSQL Windows, Npgsql.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Migration uygulanır
- ☐ Beklenen 6 tablo/index var
- ☐ Setup belgesi baştan uygulanabilir

TEST / DOĞRULAMA

dotnet ef migrations list; dotnet ef database update; dotnet ef migrations script

CODEx KOMUTU

Gün 8 - Migration ve local PostgreSQL kurulumu planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 9 / 28

Project CRUD ve persistence testleri

Faz 2
UYGULAMA PLANI

BU FAZIN HEDEFİ	İlişkisel geçmiş güvenilir ve tekrarlanabilir saklamak.
BU ADIMDA NE YAPIYORUZ?	Project create/list/get use-case'lerini EF projection ve API endpointleriyle tamamlıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	İlk uçtan uca persistence dilimi AI'dan bağımsız kanıtlanır.
BU ADIMIN MİMARİDEKİ YERİ	Application/Projects merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	ASP.NET Core, EF Core, xUnit

YAPILACAKLAR

- CreateProjectRequest'i normalize/validate et
- ProjectService ile duplicate path conflict üret
- GET list/get için AsNoTracking projection kullan
- POST/GET controller durum kodlarını bağla
- Local PostgreSQL integration testini kategori/trait ile ayır

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Projects; Api/Controllers/ProjectsController.cs; Infrastructure/Persistence/Queries; IntegrationTests/Projects

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Controller'da DbContext kullanma; fake in-memory provider ile ilişkisel davranışa güvenme.

BU ADIM BİTTİĞİNDE

Project CRUD sözleşmesi ve local DB testi çalışır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: ASP.NET Core, EF Core, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- 201 Location doğru
- Duplicate 409
- List paging bounded

TEST / DOĞRULAMA

dotnet test; yerelde özel DB varsa dotnet test --filter Category=Database

CODEX KOMUTU

Gün 9 - Project CRUD ve persistence testleri planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 10 / 28

Scanner sözleşmeleri ve manifest

Faz 3
UYGULAMA PLANI

BU FAZIN HEDEFİ	LLM'e yalnız güvenli, sınırlı ve izlenebilir repository kanıtı sağlamak.
BU ADIMDA NE YAPIYORUZ?	Scanner'ı LLM'den bağımsız typed port ve policy objeleriyle tasarlıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Scanner sonucu context ve UI için ortak güvenilir envanterdir.
BU ADIMIN MİMARİDEKİ YERİ	Application/Repositories/IRepositoryScanner.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	System.IO, options, xUnit

YAPILACAKLAR

- Extension allowlist: .cs, .csproj ve seçilmiş .json tanımla
- Directory denylist: .git, bin, obj, node_modules tanımla
- File count, per-file bytes ve total bytes limitlerini options yap
- Relative path, size, language ve skip reason içeren manifest üret
- Deterministik path sıralaması uygula

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Repositories/IRepositoryScanner.cs; ScanOptions.cs; ScanManifest.cs;
Infrastructure/Repositories/RepositoryScanner.cs

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Dosya içeriğini henüz LLM'e gönderme; gizli dosyaları sadece uzantıyla tanıma.

BU ADIM BİTTİĞİNDE

Aynı repo aynı policy ile aynı manifesti üretir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: System.IO, options, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Allowlist/denylist testli
- ☐ Limitler config edilebilir
- ☐ Manifest skipped reasons içerir

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~RepositoryScanner
```

CODEx KOMUTU

Gün 10 - Scanner sözleşmeleri ve manifest planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 11 / 28

Path, reparse point ve secret güvenliği

Faz 3
UYGULAMA PLANI

BU FAZIN HEDEFİ	LLM'e yalnız güvenli, sınırlı ve izlenebilir repository kanıtı sağlamak.
BU ADIMDA NE YAPIYORUZ?	Root confinement, canonical path kontrolü, reparse point reddi ve secret filename policy ekliyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Yerel repo path'i kullanıcı girdisidir; güven sınırındır.
BU ADIMIN MİMARİDEKİ YERİ	Infrastructure/Repositories/SafePathResolver.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	System.IO, Windows path semantics, xUnit

YAPILACAKLAR

1. Path.GetFullPath ile canonical root/target karşılaştır
2. OS'e uygun case comparison kullan
3. Root dışına çıkan .. yollarını reddet
4. Reparse point/symlink directory ve file politikasını fail-closed uygula
5. .env, key, pem, secrets ve credential adları için deny testleri yaz

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Infrastructure/Repositories/SafePathResolver.cs; SecretPathPolicy.cs; UnitTests/RepositorySecurity

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

StartsWith kontrolünü separator olmadan kullanma; linki takip etme; exception mesajında secret içerik gösterme.

BU ADIM BİTTİĞİNDE

Traversal ve link test fixture'ları güvenli biçimde reddedilir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: System.IO, Windows path semantics, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Sibling-prefix bypass engelli
- ☐ Reparse point reddedilir
- ☐ Secret files skipped

TEST / DOĞRULAMA

dotnet test --filter FullyQualifiedName~RepositorySecurity

CODEX KOMUTU

Gün 11 - Path, reparse point ve secret güvenliği planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 12 / 28

Safe file reader ve context builder

Faz 3
UYGULAMA PLANI

BU FAZIN HEDEFİ	LLM'e yalnız güvenli, sınırlı ve izlenebilir repository kanıtı sağlamak.
BU ADIMDA NE YAPIYORUZ?	İzinli dosyaları encoding kontrollü okuyup satır numaralı, bütçeli RepositoryContext'e dönüştürüyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Grounding yalnızca satırları izlenebilir context ile mümkündür.
BU ADIMIN MİMARİDEKİ YERİ	Application/Context merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	System.Text, typed results, xUnit

YAPILACAKLAR

1. Scanner manifest dışında okuma yapılmamasını sağla
2. UTF-8 strict decode, binary heuristic ve readable error result ekle
3. Her segmentte relative path ve 1-based line metadata üret
4. Global char budget ve per-file truncation marker uygula
5. Öncelik: .cs > .csproj > güvenli config olarak açık kural tanımla

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

```
Application/Context; Application/Tools/IFileReadTool.cs; Infrastructure/Tools/SafeFileReadTool.cs; UnitTests/Context
```

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Sessiz encoding replacement yapma; tam repo'yu tek stringe körce ekleme.

BU ADIM BİTTİĞİNDE

Context bütçeyi aşmaz ve her satır kaynağına geri bağlanır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: System.Text, typed results, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Budget testi geçer
- ☐ Line numbers doğru
- ☐ Read error diğer dosyaları bozmaz

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~Context
```

CODEx KOMUTU

Gün 12 - Safe file reader ve context builder planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 13 / 28

ILLMProvider ve fake provider

Faz 4
UYGULAMA PLANI

BU FAZIN HEDEFİ	Gemini'yi sağlayıcıdan bağımsız ve doğrulanabilir review akışına bağlamak.
BU ADIMDA NE YAPIYORUZ?	Application portunu sağlayıcı bağımsız, typed ve cancellation-aware kuruyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Application'ın Gemini'den bağımsızlığı test ve sağlayıcı değişimini mümkün kılar.
BU ADIMIN MİMARİDEKİ YERİ	Application/LLM/ILLMProvider.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	C# interfaces, DI, xUnit

YAPILACAKLAR

- Generic GenerateStructuredAsync<T> yerine kontrollü purpose-specific contract veya iyi sınırlandırılmış generic tasarla
- Model, prompt version, timeout metadata'sını request'e ekle
- Provider error taxonomy: timeout, rate limit, invalid response, unavailable
- Fake provider için queue/fixture tabanlı deterministic cevap oluştur
- CancellationToken'ın tüm çağrı zincirinde aktığını test et

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/LLM/ILLMProvider.cs; LLMRequest.cs; LLMResponse.cs; UnitTests/Fakes/FakeLLMProvider.cs

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Gemini DTO'larını Application'a sızdırma; gerçek HTTP çağrısı yapma.

BU ADIM BİTTİĞİNDE

Agent testleri yalnızca fake ile çalışabilir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: C# interfaces, DI, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Application Gemini paketine referans vermez
- ☐ Fake deterministic
- ☐ Cancellation testli

TEST / DOĞRULAMA

dotnet test --filter FullyQualifiedName~LLM

CODEx KOMUTU

Gün 13 - ILLMProvider ve fake provider planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 14 / 28

Prompt templates ve schema modelleri

Faz 4
UYGULAMA PLANI

BU FAZIN HEDEFİ	Gemini'yi sağlayıcıdan bağımsız ve doğrulanabilir review akışına bağlamak.
BU ADIMDA NE YAPIYORUZ?	Versioned review promptu ile CodeReviewResult/LLMFinding modellerini oluşturuyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Prompt da kod gibi sürümlenen davranış sözleşmesidir.
BU ADIMIN MİMARİDEKİ YERİ	Application/Prompts/review-system.v1.txt merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	System.Text.Json, embedded resources veya content files

YAPILACAKLAR

1. System ve task prompt rollerini ayır
2. Yalnız verilen dosya/satırlara dayanma kuralı ekle
3. Severity/category enumlarını string JSON olarak tanımla
4. Unknown property, required field, confidence ve line range validation stratejisi kur
5. PromptVersion sabitini merkezi registry'de tut

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Prompts/review-system.v1.txt; Application/Reviews/Models; Infrastructure/LLM/GeminiSchemas

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Promptları controller'a gömme; LLM DTO'yu doğrudan EF entity yapma.

BU ADIM BİTTİĞİNDE

Fixture JSON strict parse edilir; malformed örnek reddedilir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümleleriyle anlat: System.Text.Json, embedded resources veya content files.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Prompt version kaydedilebilir
- ☐ Enums string
- ☐ Invalid JSON fail-closed

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~StructuredOutput
```

CODEX KOMUTU

Gün 14 - Prompt templates ve schema modelleri planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 15 / 28

GeminiProvider HTTP adaptörü

Faz 4
UYGULAMA PLANI

BU FAZIN HEDEFİ	Gemini'yi sağlayıcıdan bağımsız ve doğrulanabilir review akışına bağlamak.
BU ADIMDA NE YAPIYORUZ?	Gemini REST çağrısını typed HttpClient, timeout, redaction ve strict parse ile uyguluyoruz.
NEDEN BU ADIMI YAPIYORUZ?	C# resmi GenAI SDK desteği listelenmediği için REST adaptörü bağımlılığı küçük ve açık tutar.
BU ADIMIN MİMARİDEKİ YERİ	Infrastructure/LLM/GeminiProvider.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	HttpClientFactory, System.Text.Json, Gemini REST

YAPILACAKLAR

- IHttpClientFactory typed client kaydı yap
- API key'i User Secrets/environment'tan al ve loglama
- Request'te response MIME type/schema yaklaşımını uygula
- 429/5xx için en fazla 2 bounded retry ve jitter politikasını değerlendir
- Fake HttpResponseMessageHandler ile success, malformed JSON, timeout ve 429 testleri yaz

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Infrastructure/LLM/GeminiProvider.cs; GeminiOptions.cs; GeminiHttpClient.cs; Tests/GeminiProviderContractTests.cs

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Her hatada retry yapma; raw prompt/response veya key loglama; normal testte internet çağırma.

BU ADIM BİTTİĞİNDE

HTTP davranışı fake handler ile tamamen testlidir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: HttpClientFactory, System.Text.Json, Gemini REST.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Timeout/cancellation çalışır
- ☐ 429 bounded retry
- ☐ Secrets logsuz

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~GeminiProvider
```

CODEx KOMUTU

Gün 15 - GeminiProvider HTTP adaptörü planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 16 / 28

Review Agent ve grounding validator

Faz 4
UYGULAMA PLANI

BU FAZIN HEDEFİ	Gemini'yi sağlayıcıdan bağımsız ve doğrulanabilir review akışına bağlamak.
BU ADIMDA NE YAPIYORUZ?	Context + prompt + provider cevabını valid Finding adaylarına çeviriyoruz.
NEDEN BU ADIMI YAPIYORUZ?	LLM çıktısı güvenilir veri değil; domain'e girmeden doğrulanır.
BU ADIMIN MİMARİDEKİ YERİ	Application/Agents/ReviewAgent.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	C#, ILLMProvider, xUnit

YAPILACAKLAR

- ReviewAgentInput'ta project, repository context ve policy tut
- Provider response'u schema doğrulamasından geçir
- FilePath manifestte yoksa finding'i reject/quarantine et
- StartLine/EndLine gerçek line count dışında ise reddet
- Duplicate findings için deterministic key ile de-dup uygula

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Agents/ReviewAgent.cs; Reviews/FindingGroundingValidator.cs; UnitTests/ReviewAgent

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Confidence yüksek diye grounding atlama; uydurulan path'i düzeltip saklama.

BU ADIM BİTTİĞİNDE

Sadece kanıtlanabilir findings domain model olur.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: C#, ILLMProvider, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Unknown file reddedilir
- ☐ Invalid line reddedilir
- ☐ Duplicate birleştirilir

TEST / DOĞRULAMA

dotnet test --filter FullyQualifiedNames~ReviewAgent

CODEX KOMUTU

Gün 16 - Review Agent ve grounding validator planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 17 / 28

Review orchestrator ve review API

Faz 4
UYGULAMA PLANI

BU FAZIN HEDEFİ	Gemini'yi sağlayıcıdan bağımsız ve doğrulanabilir review akışına bağlamak.
BU ADIMDA NE YAPIYORUZ?	Create->scan->context->agent->validate->persist->complete akışını transaction sınırlarıyla bağliyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Core demo akışı tek, gözlemlenebilir use-case olur.
BU ADIMIN MİMARİDEKİ YERİ	Application/Workflows/ReviewOrchestrator.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	ASP.NET Core, EF Core, FakeLLMProvider

YAPILACAKLAR

- ReviewRun'ı Pending/Running durumlarıyla oluştur
- Scanner/context/agent adımlarını sıralı ve max 1 model call olarak yürüt
- AIModelRun metadata ve valid findings'i atomik kaydet
- Hata durumunda ReviewRun Failed + güvenli ErrorCode yaz
- POST reviews ve GET review/history endpointlerini ekle

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Workflows/ReviewOrchestrator.cs; Api/Controllers/ReviewsController.cs; Infrastructure/Persistence; IntegrationTests/ReviewFlow

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Uzun DB transaction sırasında HTTP çağrısı tutma; failed run'ı kayıpsız bırakma.

BU ADIM BİTTİĞİNDE

Fake LLM ile uçtan uca review integration testi geçer.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: ASP.NET Core, EF Core, FakeLLMProvider.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Durum geçişleri doğru
- ☐ Finding persistence atomik
- ☐ Gemini olmadan test geçer

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~ReviewFlow
```

CODEx KOMUTU

Gün 17 - Review orchestrator ve review API planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 18 / 28

Test Agent structured output

Faz 5
UYGULAMA PLANI

BU FAZIN HEDEFİ	Test önerisi ve OS işlemlerini güvenli tool sözleşmeleriyle sunmak.
BU ADIMDA NE YAPIYORUZ?	Review bulgularından bounded test önerileri üreten ikinci agentı kuruyoruz.
NEDEN BU ADIMI YAPIYORUZ?	İkinci agent ayrı sorumluluk taşır ama yeni bir autonomous network oluşturmaz.
BU ADIMIN MİMARİDEKİ YERİ	Application/Agents/TestAgent.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	ILLMProvider, System.Text.Json, xUnit

YAPILACAKLAR

- Input'u completed ReviewRun + selected findings + ilgili context ile sınırla
- TestType: Unit, EdgeCase, Regression, Validation enumlarını kullan
- Name, target, scenario, reason zorunlu; SuggestedTestCode opsiyonel yap
- LLM'in production kod değişikliği/commit önermesini kapsam dışı belirt
- Duplicate test case ve target grounding kontrolü ekle

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Agents/TestAgent.cs; Tests/Models/TestGenerationResult.cs; Prompts/test-system.v1.txt;
UnitTests/TestAgent

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Review Agent'a test üretimini ekleyip promptu şişirme; öneriyi otomatik dosyaya yazma.

BU ADIM BİTTİĞİNDE

Fake LLM ile valid/invalid test önerileri ayrılır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümleleriyle anlat: ILLMProvider, System.Text.Json, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Yalnız completed review kabul
- ☐ Target grounded
- ☐ No source write

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~TestAgent
```

CODEX KOMUTU

Gün 18 - Test Agent structured output planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 19 / 28

Tool contracts ve audit envelope

Faz 5
UYGULAMA PLANI

BU FAZIN HEDEFİ	Test önerisi ve OS işlemlerini güvenli tool sözleşmeleriyle sunmak.
BU ADIMDA NE YAPIYORUZ?	IFileReadTool, IRepositoryScanTool ve ITestRunnerTool için typed input/output, policy ve logging sınırını ortaklaştırıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	MCP yalnızca bu güvenli servislerin dış adaptörü olacaktır.
BU ADIMIN MİMARİDEKİ YERİ	Application/Tools/*.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	C# records, logging scopes, xUnit

YAPILACAKLAR

- Her tool için success/error code/duration/truncated alanlarını tanımla
- CancellationToken ve timeout options ekle
- Input summary'yi redacted audit formatında üret
- Tool authorization/policy kontrolünü çağrıdan önce uygula
- ToolExecution kaydı için application portu ekle

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Tools/*.cs; Application/Tools/ToolResult.cs; Infrastructure/Tools; UnitTests/Tools

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Tool result'ta exception stack veya file content loglama; string command kabul etme.

BU ADIM BİTTİĞİNDE

Tüm tool'lar aynı güvenli result sözleşmesini izler.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: C# records, logging scopes, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Typed inputs
- ☐ Timeout/cancellation
- ☐ Redacted audit

TEST / DOĞRULAMA

dotnet test --filter FullyQualifiedName~Tools

CODEx KOMUTU

Gün 19 - Tool contracts ve audit envelope planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 20 / 28

Safe dotnet test runner

Faz 5
UYGULAMA PLANI

BU FAZIN HEDEFİ	Test önerisi ve OS işlemlerini güvenli tool sözleşmeleriyle sunmak.
BU ADIMDA NE YAPIYORUZ?	Sadece önceden izin verilen dotnet test operasyonunu ProcessStartInfo ile çalıştırıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Process başlatma command injection ve kaynak tüketimi açısından yüksek risklidir.
BU ADIMIN MİMARİDEKİ YERİ	Infrastructure/Tools/SafeDotnetTestRunner.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	System.Diagnostics.Process, CancellationToken, xUnit

YAPILACAKLAR

1. Executable'ı config ile güvenilir dotnet path veya PATH resolution'a bağla
2. ArgumentList ile sabit 'test' ve allowlisted project path ekle
3. UseShellExecute=false, redirect stdout/stderr, working directory=root kullan
4. Timeout'ta process tree kill ve bounded output truncation uygula
5. Arbitrary args, metacharacters, external project path testlerini reddet

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Infrastructure/Tools/SafeDotnetTestRunner.cs; TestRunnerOptions.cs; UnitTests/TestRunnerSecurity

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

cmd.exe/powershell üzerinden birleşik command çalıştırma; kullanıcıdan raw args alma.

BU ADIM BİTTİĞİNDE

Test runner güvenli fixture üzerinde sonucu structured döndürür.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümleleriyle anlat: System.Diagnostics.Process, CancellationToken, xUnit.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Shell kullanılmaz
- ☐ Traversal/args reddedilir
- ☐ Timeout process'i bitirir

TEST / DOĞRULAMA

dotnet test --filter FullyQualifiedName~TestRunnerSecurity

CODEX KOMUTU

Gün 20 - Safe dotnet test runner planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 21 / 28

Test orchestration ve API

Faz 5
UYGULAMA PLANI

BU FAZIN HEDEFİ	Test önerisi ve OS işlemlerini güvenli tool sözleşmeleriyle sunmak.
BU ADIMDA NE YAPIYORUZ?	Test Agent önerilerini kaydedip opsiyonel safe run akışını API'ye bağlıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Öneri üretmek ve test çalıştırmak ayrı yetki/yan etki seviyeleridir.
BU ADIMIN MİMARİDEKİ YERİ	Application/Workflows/TestOrchestrator.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	ASP.NET Core, EF Core, FakeLLMProvider, tools

YAPILACAKLAR

1. POST /api/reviews/{id}/tests endpointini ekle
2. Review ve findings'i AsNoTracking yükle
3. TestAgent çağrısı ve AIModelRun metadata kaydını yap
4. TestCase kayıtlarını transaction ile ekle
5. Ayrı test-runs endpointini explicit kullanıcı aksiyonuna bağla

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Workflows/TestOrchestrator.cs; Api/Controllers/ReviewsController.cs; IntegrationTests/TestFlow

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Test üretince otomatik process başlatma; aynı review için kontrolsüz tekrar üretme.

BU ADIM BİTTİĞİNDE

Fake ile test önerisi ve fake runner ile run API testleri geçer.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümleleriyle anlat: ASP.NET Core, EF Core, FakeLLMProvider, tools.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Tests endpoint 201
- ☐ Runner explicit action
- ☐ ToolExecution kaydı

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~TestFlow
```

CODEX KOMUTU

Gün 21 - Test orchestration ve API planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 22 / 28

MCP server adapter başlangıcı

Faz 6
UYGULAMA PLANI

BU FAZIN HEDEFİ	Çalışan tool'ları MCP ile standartlaştırıp workflow sınırlarını sertleştirmek.
BU ADIMDA NE YAPIYORUZ?	Resmi C# MCP SDK ile mevcut read_file tool'unu ince bir adapter olarak expose ediyoruz.
NEDEN BU ADIMI YAPIYORUZ?	MCP güvenliği sağlamaz; yalnızca zaten güvenli tool'u standartlaştırır.
BU ADIMIN MİMARİDEKİ YERİ	Infrastructure/Mcp merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	Official ModelContextProtocol C# SDK, DI

YAPILACAKLAR

- Önce stdio mu ASP.NET HTTP mi seçip ADR yaz; portföy/local için stdio varsayılanını değerlendir
- ModelContextProtocol NuGet paketini yalnız MCP katmanına ekle
- read_file MCP argümanlarını SafeFileReadTool input'una map et
- Root repository id/path authorization bağlamını koru
- Protocol output'ta raw exception değil structured error döndür

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Infrastructure/Mcp; Api veya ayrı CodeGuardAI.McpHost değerlendirmesi; docs/adr/003-mcp-host.md; Tests/Mcp

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

MCP handler içinde yeni filesystem kodu yazma; tool'u bypass etme.

BU ADIM BİTTİĞİNDE

MCP read_file ve doğrudan tool aynı security testlerini paylaşır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: Official ModelContextProtocol C# SDK, DI.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Tek source of truth tool service
- ☐ MCP smoke test
- ☐ Application SDK'ya bağımlı değil

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~Mcp
```

CODEX KOMUTU

Gün 22 - MCP server adapter başlangıcı planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 23 / 28

run_tests MCP tool ve sınırlar

Faz 6
UYGULAMA PLANI

BU FAZIN HEDEFİ	Çalışan tool'ları MCP ile standartlaştırıp workflow sınırlarını sertleştirmek.
BU ADIMDA NE YAPIYORUZ?	İkinci ve son ilk-sürüm MCP tool olarak kontrollü test runner'ı expose ediyoruz.
NEDEN BU ADIMI YAPIYORUZ?	MCP yüzeyi küçük tutulursa security review yapılabilir.
BU ADIMIN MİMARİDEKİ YERİ	Infrastructure/Mcp/RunTestsMcpTool.cs merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	MCP C# SDK, SafeDotnetTestRunner

YAPILACAKLAR

- Input schema'yı repository/project id ve allowlisted target ile sınırla
- MCP cancellation token'ı runner'a geçir
- Timeout/output limitlerini server değil shared options üzerinden uygula
- ToolExecution audit kaydını koru
- Başarısız test ile tool infrastructure error ayrımını map et

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Infrastructure/Mcp/RunTestsMcpTool.cs; Tests/Mcp/RunTestsMcpToolTests.cs

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Shell command alanı ekleme; modelden gelen working directory'ye güvenme.

BU ADIM BİTTİĞİNDE

MCP üzerinden yalnız izinli test işlemi çalışır.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: MCP C# SDK, SafeDotnetTestRunner.

KONTROL / ACCEPTANCE CRITERIA

- ☐ No arbitrary command
- ☐ Timeout propagated
- ☐ Audit record created

TEST / DOĞRULAMA

dotnet test --filter FullyQualifiedName~RunTestsMcp

CODEX KOMUTU

Gün 23 - run_tests MCP tool ve sınırlar planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 24 / 28

Deterministik orchestrator hardening

Faz 6
UYGULAMA PLANI

BU FAZIN HEDEFİ	Çalışan tool'ları MCP ile standartlaştırıp workflow sınırlarını sertleştirmek.
BU ADIMDA NE YAPIYORUZ?	Review ve test workflow'larına max steps, idempotency ve concurrency koruması ekliyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Agentic davranış kontrollü workflow olarak kalmalı; sonsuz agent loop olmamalı.
BU ADIMIN MİMARİDEKİ YERİ	Application/Workflows merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	C#, EF concurrency, logging

YAPILACAKLAR

1. Review akışında sabit state machine ve max step sayısı tanımla
2. Aynı project için duplicate concurrent review politikasını uygula
3. Retry yalnız provider'ın transient sınıfında ve orchestrator dışı bounded policy'de olsun
4. Correlation/ReviewRunId logging scope ekle
5. Yarım kalan run recovery davranışını belge

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Application/Workflows; Infrastructure/Persistence/Concurrency; IntegrationTests/WorkflowSafety

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Agentların birbirini çağırmasına izin verme; recursive tool loop oluşturma.

BU ADIM BİTTİĞİNDE

Akış terminal durumunda biter ve eşzamanlı istek deterministic yanıt verir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: C#, EF concurrency, logging.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Max steps enforcement
- ☐ Duplicate conflict
- ☐ Terminal state persisted

TEST / DOĞRULAMA

```
dotnet test --filter FullyQualifiedName~WorkflowSafety
```

CODEx KOMUTU

Gün 24 - Deterministik orchestrator hardening planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 25 / 28

Basit frontend ve demo akışı

Faz 7
UYGULAMA PLANI

BU FAZIN HEDEFİ	Ürünü kullanılabilir, ölçülebilir ve portföye hazır hale getirmek.
BU ADIMDA NE YAPIYORUZ?	Frameworksüz UI ile project, scan, review, findings filtreleri ve test önerisi akışını görünür yapıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Portföy demosu backend'i erişilebilir kılar ama ayrı SPA yükü yaratmaz.
BU ADIMIN MİMARİDEKİ YERİ	Api/wwwroot/index.html merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	HTML, CSS, JavaScript fetch

YAPILACAKLAR

1. Project create/list ekranı yap
2. Repository path inputunun server-local path olduğunu açıkça belirt
3. Scan özetini included/skipped counts ile göster
4. Review progress/result ve severity/category filtreleri ekle
5. Generate Tests butonunu ayrı explicit aksiyon yap

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

Api/wwwroot/index.html; styles.css; app.js; Api static files config

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

React/build chain ekleme; browser'dan OS folder picker varmış gibi davranma; secret gösterme.

BU ADIM BİTTİĞİNDE

Ana demo flow tek sayfadan yürütülebilir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: HTML, CSS, JavaScript fetch.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Loading/error/empty states
- ☐ Accessible labels
- ☐ API errors readable

TEST / DOĞRULAMA

dotnet run; manuel demo checklist; dotnet test

CODEX KOMUTU

Gün 25 - Basit frontend ve demo akışı planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 26 / 28

Eval ve observability

Faz 7

UYGULAMA PLANI

BU FAZIN HEDEFİ	Ürünü kullanılabilir, ölçülebilir ve portföye hazır hale getirmek.
BU ADIMDA NE YAPIYORUZ?	Sabit kod fixture'larıyla davranış eval'i ve structured metrics/log kapsamını tamamlıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	LLM kalitesi sadece compile-time testle ölçülemez; eval regresyon sinyali verir.
BU ADIMIN MİMARİDEKİ YERİ	tests/CodeGuardAI.EvalTests veya UnitTests/Evals merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	xUnit, FakeLLMProvider, Microsoft.Extensions.Logging

YAPILACAKLAR

- 3-5 küçük fixture ekle: null, validation, path traversal, resource disposal, clean control
- Expected category/file/valid-line gibi semantic assertions tanımla
- Exact prose karşılaştırma yapma
- ReviewRunId/ProjectId/model/prompt/duration/finding count log eventlerini standardize et
- Log redaction testleri ekle

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

```
tests/CodeGuardAI.EvalTests veya UnitTests/Evals; tests/Fixtures; Application/Observability; docs/evaluation.md
```

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Gerçek Gemini'yi normal suite'e bağlama; prompt/secret/repo içeriğini loglama.

BU ADIM BİTTİĞİNDE

Eval offline koşar ve metrik sözlüğü belgelenir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: xUnit, FakeLLMProvider, Microsoft.Extensions.Logging.

KONTROL / ACCEPTANCE CRITERIA

- 5 fixture veya gerekçeli 3+
- Semantic assertions
- Sensitive log testleri

TEST / DOĞRULAMA

```
dotnet test --filter Category=Eval; dotnet test
```

CODEX KOMUTU

Gün 26 - Eval ve observability planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 27 / 28

GitHub Actions ve tam doğrulama

Faz 7
UYGULAMA PLANI

BU FAZIN HEDEFİ	Ürünü kullanılabilir, ölçülebilir ve portföye hazır hale getirmek.
BU ADIMDA NE YAPIYORUZ?	Container olmadan restore-build-test pipeline'ı ve local DB test ayrımını kuruyoruz.
NEDEN BU ADIMI YAPIYORUZ?	CI temel kaliteyi her push/PR'da kanıtlar; container yasağını ihlal etmez.
BU ADIMIN MİMARİDEKİ YERİ	.github/workflows/ci.yml merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	GitHub Actions, dotnet CLI

YAPILACAKLAR

- actions/checkout ve setup-dotnet ile .NET 10 pinle
- dotnet restore, build --no-restore, test --no-build adımlarını ekle
- Unit/API/fake LLM testlerini CI'da offline çalıştır
- Database tests için Category=Database ve local-only dokümantasyonu koru
- Test result artifact ve cache'i gereksizse ekleme; minimum pipeline tut

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

.github/workflows/ci.yml; docs/testing.md

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Postgres service container ekleme; Gemini key secret talep etme; continue-on-error kullanma.

BU ADIM BİTTİĞİNDE

CI temiz runner'da secretsiz yeşil olur.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: GitHub Actions, dotnet CLI.

KONTROL / ACCEPTANCE CRITERIA

- ☐ No Docker/service container
- ☐ No Gemini call
- ☐ Build/test fail pipeline

TEST / DOĞRULAMA

Workflow syntax review; GitHub run; local dotnet test

CODEX KOMUTU

Gün 27 - GitHub Actions ve tam doğrulama planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

GÜN 28 / 28

Dokümantasyon, demo ve portföy kapanışı

Faz 7
UYGULAMA PLANI

BU FAZIN HEDEFİ	Ürünü kullanılabilir, ölçülebilir ve portföye hazır hale getirmek.
BU ADIMDA NE YAPIYORUZ?	README, diyagramlar, setup, güvenlik notları, ekran görüntüleri ve gerçekçi CV maddelerini tamamlıyoruz.
NEDEN BU ADIMI YAPIYORUZ?	Portföy değeri yalnız koddan değil, kararların anlatılabilirliğinden gelir.
BU ADIMIN MİMARİDEKİ YERİ	README.md merkezli; bağımlılık yönü ve sonraki günün giriş sözleşmesini hazırlar.
KULLANILACAK TEKNOLOJİLER	Markdown, GitHub, screenshots

YAPILACAKLAR

1. README'e problem, architecture, quickstart ve limitations ekle
2. Architecture/ER/AI/MCP diyagramlarını güncel kodla karşılaştıır
3. Local Windows PostgreSQL ve User Secrets kurulumunu sıfırdan test et
4. Demo script ve 2-4 ekran görüntüsü/GIF planı hazırla
5. Yalnız tamamlanan teknolojilerden CV maddeleri yaz ve final checklist'i işaretle

OLUŞTURULACAK / DEĞİŞTİRİLECEK DOSYALAR

README.md; docs/architecture.md; docs/security.md; docs/demo-script.md; screenshots/; sample-repository/

DİKKAT EDİLMESİ GEREKENLER

Değişiklik küçük ve geri alınabilir kalmalı; public contract, dependency direction, cancellation ve error davranışı görünür olmalı. Test, yalnız başarılı yolu değil bugünün ana riskini de kanıtlamalı.

YAPILMAMASI GEREKENLER

Yapılmamış özelliği completed gösterme; Docker sözü ekleme; kırmızı CI ile bitirme.

BU ADIM BİTTİĞİNDE

Yeni biri README ile çalıştırabilir ve 5 dakikalık demo yapılabilir.

BU ADIMDA ÖĞRENİLECEKLER

Bu günün sonunda yalnız API kullanımını değil, seçimin trade-off'unu, katman sınırını ve başarısızlık davranışını açıklayabilmelisin. Özellikle şu teknolojik odağı kendi cümlelerinle anlat: Markdown, GitHub, screenshots.

KONTROL / ACCEPTANCE CRITERIA

- ☐ Fresh setup doğrulandı
- ☐ Tüm diagramlar gerçekle uyumlu
- ☐ CV maddeleri kanıtlanabilir

TEST / DOĞRULAMA

dotnet build; dotnet test; manuel demo; git status ve secret scan kontrolü

CODEX KOMUTU

Gün 28 - Dokümantasyon, demo ve portföy kapanışı planını uygula. Codex Genel Uygulama Kuralları'na ve yukarıdaki kapsam, yasaklar, hedef dosyalar, test/doğrulama adımları ile acceptance criteria'ya eksiksiz uy. Yalnız bu görev için gerekli repository dosyalarını incele; acceptance criteria sağlandığında dur ve sonraki güne geçme.

32. Proje Tamamlanma Kriterleri

- Backend tamamen C#/.NET 10 ve ASP.NET Core'dur; production solution'da Python yoktur.
- Docker/container dosyası, komutu veya CI service container'ı yoktur.
- Local Windows PostgreSQL setup sıfırdan uygulanabilir ve EF migration günceldir.
- Project, ReviewRun, Finding, TestCase, AIModelRun ve ToolExecution modeli API ile uyumludur.
- Scanner secret/path/reparse/size kurallarını testlerle uygular.
- Application Gemini'ye değil ILLMProvider'a bağlıdır.
- Normal dotnet test gerçek Gemini çağrısı yapmaz ve key istemez.
- Structured JSON, field validation ve file/line grounding geçmeden DB'ye girmez.
- Test Agent production kodunu değiştirmez; runner raw shell kabul etmez.
- MCP yalnız çalışan safe tools üzerine ince adapter olarak eklenmiştir.
- Workflow bounded ve terminal state üretir; agent loop yoktur.
- UI ana demo akışını yürütür; history ve filtreler çalışır.
- CI restore/build/test çalıştırır; container ve Gemini secret gerektirmez.
- README, diagrams, setup, security, eval ve limitations günceldir.

33. Final Ürün ve Teknik Demo Flow

1. ASP.NET Core host local olarak dotnet run ile açılır; static UI aynı origin'den servis edilir.
2. POST /api/projects normalized path'i doğrular ve Project kaydını PostgreSQL'e yazar.
3. Scan endpointi root-bound scanner'ı çalıştırır; manifest included/skipped nedenleriyle döner.
4. Review endpointi ReviewRun oluşturur; orchestrator context budget uygular.
5. Review Agent versioned prompt ve schema ile ILLMProvider'ı çağırır.
6. GeminiProvider typed HttpClient üzerinden Gemini REST API'ye gider; metadata AIModelRun'a yazılır.
7. JSON strict deserialize edilir; enum/range ve file/line grounding uygulanır.
8. Valid Finding kayıtları transaction ile saklanır; ReviewRun Completed olur.
9. UI bulguları severity/category/file/line/reason/suggestion/confidence ile filtreler.
10. Generate Tests ayrı endpointi Test Agent'a completed review + ilgili context verir.
11. TestCase önerileri saklanır; production code değiştirilmez.
12. Kullanıcı ayrıca Run Tests isterse ITestRunnerTool/MCP adapter yalnız allowlisted dotnet test çalıştırır.
13. ToolExecution bounded output ve duration ile audit edilir; eski review'lar yeniden görüntülenir.

34. GitHub Repository Checklist

Item	Acceptance Criteria
README.md	Problem, demo, architecture, stack, quickstart, limitations, screenshots
docs/architecture.md	Dependency diagram, review/test flow, ADR linkleri
docs/database.md	ER diagram, migration ve relationship açıklaması
Configuration	appsettings örneği, User Secrets/env komutları; gerçek secret yok
Setup	.NET 10, local Windows PostgreSQL, DB/user/migration, dotnet run/test
Testing	Unit/integration/database/eval kategorileri ve offline davranış
Sample repo	Küçük, lisansı açık veya projeye ait fixture; secret yok
Media	2-4 screenshot; 60-90 saniyelik GIF/video önerisi
CI	GitHub Actions restore/build/test; Docker ve Gemini key yok
API docs	OpenAPI kullanım adresi ve endpoint tablosu
Security	Threat model, path/tool/prompt/secret kontrolleri ve limitations
Decisions	ADR: modular monolith, controllers, no generic repository, MCP timing

35. CV'de Nasıl Gösterilebilir?

Kullanım kuralı

Aşağıdaki maddeleri ancak ilgili acceptance criteria gerçekten tamamlandıysa kullan. MCP veya Gemini eksiğe o ifadeyi çıkar. Docker hiçbir varyantta eklenmez.

- Built a modular-monolith code review assistant with C#, ASP.NET Core, EF Core and PostgreSQL, persisting schema-validated, file/line-grounded findings through a tested REST API.
- Integrated Gemini behind an ILLMProvider abstraction with versioned prompts, structured JSON validation, bounded context construction and deterministic fake-provider tests.
- Designed security-constrained repository and test tools with path traversal, secret exposure and command injection defenses; exposed selected tools through the official MCP C# SDK and automated offline build/tests with GitHub Actions.

36. Future Improvements

Bu maddeler 28 günlük MVP'ye dahil değildir; ihtiyaç kanıtlandığında ayrı ADR ve threat model ile ele alınır.

Item	Acceptance Criteria	Future Impact / Risk
GitHub repository / PR integration	Local MVP stabil ve OAuth/webhook ihtiyacı netleşirse	Auth, rate limit ve external side effects kapsamı büyütür
PR review comments	False positive oranı ölçülüp human approval tasarlanırsa	Yanlış yorum dış kullanıcıyı etkiler
Authentication	Remote deployment veya çok kullanıcı olursa	Local tek kullanıcı için gereksiz
RAG / pgvector	Çok büyük repo ve cross-run knowledge ihtiyacı ölçülürse	Önce deterministic context baseline gerekir
Langfuse/advanced eval	Prompt sayısı ve ekip büyürse	Zorunlu SaaS ve ek operasyon yükü
Multiple providers	Gemini quota/quality ihtiyacı kanıtlanırsa	ILLMProvider zaten extension seam sağlar
Advanced MCP tools	Mevcut iki tool güvenlik incelemesinden geçerse	Yüzey büyüdükçe risk artar
Sandbox isolation	Untrusted test execution gerçek kullanım haline gelirse	Firecracker/gVisor/Kubernetes öğrenci MVP'si için ağır
Containerization/deployment	Local-first hedef değişirse	Bu proje sürümünde açıkça yasak ve gerekli değil

37. Son Teknik Tutarlılık Kontrolü

Kontrol	Sonuç
C#/ .NET only application	Evet: ASP.NET Core, EF Core, xUnit; Python production planında yok.
Docker/container	Yok: local Windows PostgreSQL ve secretsiz offline CI.
API - database uyumu	Project ve ReviewRun endpointleri 6 entity/relationship modeliyle uyumlu.
DTO/domain/entity ayrımı	Boundary, untrusted LLM ve persistence rolleri ayrılmış.
Scanner - Review Agent	ScanManifest ve RepositoryContext agent inputunu doğrudan sağlar.
Test Agent data	Completed review, findings ve target context mevcuttur.
Tool before MCP	Gün 19-21 tools; gün 22-23 MCP adapter.
Provider inversion	Application ILLMProvider'a; Gemini Infrastructure'a bağlı.
Test isolation	FakeLLMProvider ve fake HTTP; real Gemini yalnız opt-in smoke.
Security	Traversal, reparse, secret, prompt injection, command injection, JSON ve grounding ele alınmış.
Dependency order	Her gün yalnız önceki çıktılarına dayanır; MCP ve UI sona yakın.
Scope realism	İki agent, dört production project, basit UI; microservice/event bus yok.

38. Kaynaklar ve Sürüm Notu

- Microsoft .NET Support Policy - <https://dotnet.microsoft.com/en-us/platform/support/policy>
- ASP.NET Core Web API documentation - <https://learn.microsoft.com/aspnet/core/web-api/?view=aspnetcore-10.0>
- EF Core 10 What's New - <https://learn.microsoft.com/ef/core/what-is-new/ef-core-10.0/whatsnew>
- Gemini API libraries - <https://ai.google.dev/gemini-api/docs/libraries>
- Official MCP C# SDK - <https://github.com/modelcontextprotocol/csharp-sdk>

Sürüm notu: Bu rehber 29 Eylül 2026 tarihindeki resmi destek bilgilerine göre .NET 10 LTS ve EF Core 10 seçer. Uygulama başlanırken global.json ve NuGet sürümleri resmi stabil patch sürümlerine pinlenmeli; preview paketlerden kaçınılmalıdır.

Son söz

Başarı ölçütü 28 günde çok teknoloji kullanmak değil, her katmanın neden var olduğunu gösterebilen, güvenlik sınırları test edilmiş ve baştan sona demo edilebilen küçük ama ciddi bir ürün ortaya çıkarmaktır.